

What changes after deployment? A survey on On-device Learning in TinyML

MASSIMO PAVAN*, Technical University of Denmark (DTU), Denmark

LUCA PEZZAROSSA, Technical University of Denmark (DTU), Denmark

FABRIZIO PITTORINO, Politecnico di Milano, Italy

MANUEL ROVERI, Politecnico di Milano, Italy

XENOFON FAFOUTIS, Technical University of Denmark (DTU), Denmark

Machine learning models on microcontroller-class devices (TinyML) face a fundamental challenge: post-deployment distribution change undermines static models. On-device learning (ODL) addresses this by running the learning process directly on the device. The existing literature has not characterized how distribution change occurs or how different change types require different solutions. Approximately 70 ODL works are surveyed under one principle: the distribution change regime. The survey analyzes how different types of distribution change influence the applications addressable on-device, the hardware employed, and the structure of the solutions. A persistent gap between methodological benchmarks and real-world deployment scenarios is also identified.

CCS Concepts: • **Computer systems organization** → *Embedded systems*; • **General and reference** → **Surveys and overviews**; • **Computing methodologies** → *Machine learning*; **Learning settings**.

Additional Key Words and Phrases: TinyML, on-device learning, on-device training, microcontrollers, distribution shift, concept drift, continual learning, edge intelligence, transfer learning

ACM Reference Format:

Massimo Pavan, Luca Pezzarossa, Fabrizio Pittorino, Manuel Roveri, and Xenofon Fafoutis. 2026. What changes after deployment? A survey on On-device Learning in TinyML. 1, 1 (May 2026), 32 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

The last decade has witnessed an unprecedented proliferation of intelligent embedded devices, from wearables monitoring vital signs to industrial sensors detecting early signs of mechanical failure. At the heart of this trend lies TinyML: the discipline of deploying machine learning algorithms directly on highly constrained hardware, such as microcontrollers (MCUs), operating under tight budgets of memory, compute, and energy [96]. By pushing ML inference to the extreme edge of the network (i.e., into the devices themselves, rather than delegating computation to remote servers) TinyML enables applications that are responsive, privacy-preserving, and functional even in the absence of network connectivity [4]. Historically, the dominant paradigm in TinyML has been to separate the learning and inference phases entirely:

*This project has received funding from the European Union's Horizon Europe research and innovation programme under the HORIZON-JU-Chips-2024-1-IA grant agreement No 101194172. The project is also supported by its members France, Italy, Czech Republic, Germany, Sweden, Denmark, Greece, Spain, Netherlands, Portugal, Turkey, Switzerland. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or CHIPS JU. Neither the European Union nor the granting authority can be held responsible for them.

Authors' Contact Information: Massimo Pavan, mapav@dtu.dk, Technical University of Denmark (DTU), Kongens Lyngby, Denmark; Luca Pezzarossa, lpez@dtu.dk, Technical University of Denmark (DTU), Kongens Lyngby, Denmark; Fabrizio Pittorino, Politecnico di Milano, Milan, Italy; Manuel Roveri, Politecnico di Milano, Milan, Italy; Xenofon Fafoutis, xefa@dtu.dk, Technical University of Denmark (DTU), Kongens Lyngby, Denmark.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Manuscript submitted to ACM

models are trained on external hardware and subsequently deployed to the device, which is then responsible solely for running predictions. This separation keeps the computational burden on the device to a minimum, but it comes at a cost. Models trained offline on curated datasets frequently underperform once exposed to real-world conditions, as the data encountered after deployment rarely matches the distribution seen during training [47, 98]: in other words, the data distribution changes after deployment. These shortcomings have motivated a growing body of work under the umbrella of *On-Device Learning* (ODL) [67], which seeks to execute not just inference but the learning process itself on the device, with the objective of adapting models to the distribution changes that occur after deployment.

Realizing this vision on TinyML hardware, however, requires confronting constraints with no equivalent in conventional machine learning: memory and compute budgets must now accommodate the learning process alongside inference, labeled data is scarce and costly to obtain, and models cannot be validated before they start executing inference. Despite these constraints, ODL substantially widens the scope of what TinyML systems can do: it enables fine-tuning to specific deployment conditions [28], personalization to individual users or contexts [51], and continuous adaptation to non-stationary data streams [18, 23]. The development of adaptive TinyML solutions is therefore an increasingly active research area, and one in need of careful analysis. However, despite the centrality of distribution change as the fundamental motivation for ODL, the existing literature has devoted surprisingly little effort to characterizing the different types of change that can occur after deployment, and prior surveys do not make explicit that different solutions operate under fundamentally different distribution change regimes. Motivated by this observation, we present a comprehensive survey of the Tiny ODL literature that, for the first time, places the nature of the distribution change at the center of its structure and analysis, using it as the primary lens through which solutions are characterized, compared, and evaluated.

Concretely, we distinguish three types of distribution change regimes. In the *single-change* regime, the distribution shifts exactly once at deployment and then remains stable: the challenge is to adapt quickly and efficiently to the new conditions, so that effective inference can be performed for the rest of the device’s life. In the *concept drift* regime, the distribution shifts continuously and unpredictably throughout the operational life of the device, requiring the solution to adapt quickly to each new condition while discarding knowledge that is no longer relevant. In the *continual learning* regime, new concepts are introduced incrementally over time, and the solution must accumulate new knowledge without forgetting what it has previously learned, a challenge known as catastrophic forgetting [18]. Together, these three regimes cover all distribution change scenarios addressed in the ODL literature surveyed in this work: every surveyed solution can be categorized as working under one of them.

These three settings place fundamentally different demands on the applications that can be addressed and on the learning algorithm and the hardware we use to address them, and important patterns only emerge when works are compared within the same regime. For each regime, we analyze the literature across three complementary lenses: the *application* contexts each regime naturally accommodates, the *hardware* platforms suited to its computational demands, and the *technical* composition of the solutions it requires. Beyond characterizing each regime individually, this structure exposes patterns that cut across all three regimes: first, which application domains tend to drive the use of particular hardware platforms, and second, which hardware constraints rule out entire families of learning algorithms, leaving only a narrow set of viable approaches for a given device.

The rest of the paper is organized as follows. Section 2 positions the survey with respect to prior work. Section 3 provides a formal definition of the ODL problem, including the data-generating process, the device constraints, the components of an ODL solution and the evaluation methodology. Sections 4, 5 and 6, present the application, hardware,

Table 1. Comparison of related surveys on on-device learning for TinyML. Taxonomy abbreviations: **NN** = Neural Networks, **DT** = Decision Trees. Aspects column: **M** = Models, **A** = Application, **HW** = Hardware.

Survey	Year	Tiny Devices	Taxonomy	Aspects	#ODL Works
Rajapakse et al. [67]	2023	✓	offline v online	M, A	~10
Abozaid et al. [2]	2025	✓	–	M	~10
Lourenço et al. [46]	2025	✗	NN v DT	M	~10
This survey	2026	✓	Distr. Change	M, A, HW	~70

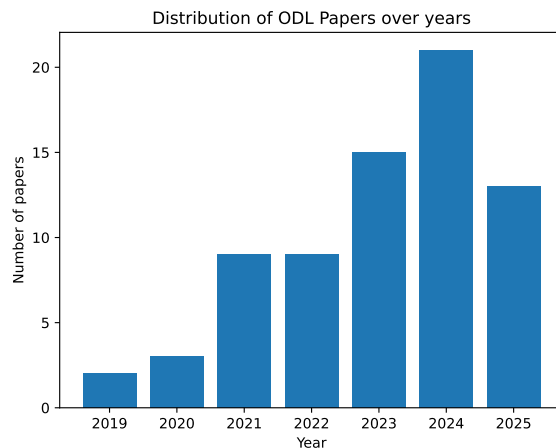


Fig. 1. Number of papers published over the years in ODL for TinyML.

and technical lenses respectively. Section 7 discusses open challenges and future directions, and Section 8 concludes the paper.

2 Scope and Positioning

Several surveys have recently addressed adjacent topics, but none covers the same combination of scope and perspective: prior works either target a broader hardware range than MCU-class devices or organize solutions along a technical axis that cuts across the distribution change regimes introduced here. Table 1 summarizes the key differences between related surveys and the present work.

Rajapakse et al. [67] is the closest in spirit, surveying both Over-The-Air (OTA) updates and ODL approaches for TinyML. Their taxonomy organizes ODL solutions along a technical axis, distinguishing offline learning from online learning approaches. While useful for characterizing algorithmic families, this axis cuts across the distribution change regime a solution operates under: a solution in which the distribution changes only once at the moment of deployment and one in which it can continuously change are grouped together, despite solving fundamentally different problems under fundamentally different constraints. Furthermore, the survey was published in 2022 and does not capture the substantial body of work published since. As visible in Figure 1, most of the works present in the literature were published after 2023, highlighting the growing relevance of the field. Other more recent works are present in the

literature, but they are less close in scope with respect to [67]. Abozaid et al. [2] focus narrowly on the model-related aspects, offering detailed accounts of individual papers without a unifying taxonomic framework. Lourenço et al. [46] restrict their scope to data stream learning for IoT, covering neural networks and decision trees as relevant model families, but without a focus on truly constrained hardware and without considering the hardware and application aspects.

The present survey differs from prior work in three respects. First, it is strictly scoped to MCU-class hardware and below, keeping hardware constraints central to the analysis. Second, with approximately 70 works surveyed, it covers a substantially larger portion of the literature than any prior effort. Third, and most distinctively, it organizes the entire literature around the distribution change regime, distinguishing between single-change, concept drift, and continual learning regimes, and analyzes each through three complementary lenses: application, hardware, and technical composition.

2.1 Literature Search and Inclusion Criteria

The literature search was conducted using Google Scholar as the primary database, complemented by IEEE Xplore, ACM Digital Library, and arXiv. The search combined several keyword groups, including “On-Device Learning”, “On-Device Training”, “Online Learning”, “Continual Learning”, “Transfer Learning”, and “Learning in the presence of concept drift”, each paired with the terms “tiny”, “MCU”, or “IoT” to restrict results to the TinyML domain. The references of selected works were iteratively reviewed to identify additional relevant contributions not captured by the initial keyword search.

Works were included in the survey if they satisfied the following criteria:

- The work targets hardware within the TinyML scope. Concretely, this corresponds to MCU-class devices (e.g., ARM Cortex-M series) or below, operating under tight M_D (in the order of the MB), F_D (<500 MHz), and power (<500 mW) budgets. Application-class processors (e.g., ARM Cortex-A series, as found in devices such as the Raspberry Pi) were considered outside this scope, unless the work explicitly frames its solution around the constraints of MCU-class deployment.
- The work addresses the learning phase of a machine learning algorithm, not inference only.
- The learning phase is executed or designed to be executed on the target device, not exclusively on external hardware.

3 Problem Formulation for On-device Learning under distribution changes

This section formalizes the on-device learning problem under distribution change, establishes the vocabulary and conceptual framework used throughout, and defines the three distribution change regimes that categorize the surveyed works across the application, hardware, and technical lenses.

The goal of the ODL problem is to obtain a solution A^1 addressing an application, which comprises and enables the on-device inference and learning of a machine learning model over data drawn from a data-generating process P directly on a target device D . The nature of P , the requirements of the application, and the constraints of D together determine the space of viable solutions, and are formally characterized in the following subsections.

Section 3.1 characterizes P and introduces the distribution change regime taxonomy that serves as the primary organizing principle of this survey: the nature of the change in P after T determines which regime a solution operates under.

¹We adopt the term *ODL solution* rather than *ODL model*, as used in some prior work, to emphasize that a complete ODL solution must encompass not only the components for model learning and inference, but also those responsible for extracting and managing relevant features from the incoming data stream.

Section 3.2 formalizes the requirements that an application may impose on the deployment. Section 3.3 characterizes D and the constraint it imposes on A . Section 3.4 introduces the component decomposition of A used throughout the analysis. The criteria for evaluating A are presented in Section 3.5.

3.1 The data generating process P and the distribution change regimes

Let P be a data generating process that, at each time instant t , provides a pair (x^t, y^t) sampled from an unknown probability distribution $p^t(x, y)$, where x is the data collected from an on-device sensor (e.g., an image or an audio clip) and y its classification label. Without loss of generality, the supervised information y^t might not be available at every time instant, reflecting the practical reality of TinyML deployments where labeled data is scarce and the device spends the majority of its operational life executing inference rather than learning.

We assume that there exists a time instant T when the proposed solution is deployed on an embedded device D . We assume that before T , the unknown distribution is stationary: $\forall t < T: p^t(x, y) = p^{t+1}(x, y)$. This property might not hold for $t \geq T$, since the unknown distribution might change after deployment. It is worth noting that the change in $p^t(x, y)$ might affect the input x (e.g., by the introduction of noise), the set of classes (e.g., class change), or the relationship between x and y . The set of realizations $S_0 = ((x^0, y^0), \dots, (x^{T-1}, y^{T-1}))$ obtained from P before deployment constitutes a dataset that can be used for pre-training or initializing the various components of the solution A , while the set of realizations $S = ((x^T, y^T), \dots)$ represents a stream of data arriving at the device after deployment, which can be used for the adaptation of the solution.

3.1.1 Distribution change regimes. Depending on whether and when a change in the distribution of P occurs during S , we can distinguish among three distribution change regimes:

- *Single-change regime:* The distribution of the data-generating process changes exactly once, at the moment of deployment T , i.e., $p^{T-1}(x, y) \neq p^T(x, y)$ and $\forall t > T: p^t(x, y) = p^{t+1}(x, y)$. Note that this scenario also encompasses solutions trained entirely on-device without any pre-deployment data, i.e., solutions with $S_0 = \emptyset$. A typical example is a keyword spotting system that enrolls a user-defined wake word at deployment time and requires no further adaptation thereafter [73].
- *Concept drift regime:* The distribution of the data-generating process changes multiple times after deployment, i.e., $\exists t > T: p^{t-1}(x, y) \neq p^t(x, y)$. Changes are typically frequent and the focus is on designing solutions that adapt quickly to the new distribution, discarding or downweighting knowledge acquired under previous distributions. A typical example is a pressure sensor calibration model that continuously tracks measurement drift caused by environmental changes [83].
- *Continual learning regime:* This regime shares the same formal definition as the concept drift regime: $\exists t > T: p^{t-1}(x, y) \neq p^t(x, y)$. The distinction is not in the properties of P , but in the assumptions made about the relevance of past knowledge and in the criteria used to evaluate solutions. Changes in this type of distribution are typically less frequent and correspond to the introduction of genuinely new concepts or tasks. The focus is therefore on designing solutions that acquire new knowledge without forgetting previously learned information, a challenge commonly referred to as catastrophic forgetting [56]. A typical example is an EEG-based brain-machine interface that incrementally learns to recognize new motor commands while retaining the ones learned previously [52].

While the concept drift and continual learning regimes share the same formal characterization of P , treating them as a single category would group solutions designed under incompatible assumptions about the relevance of past

knowledge, making direct comparison between them meaningless. The distinction is therefore maintained as a primary organizing principle throughout this survey.

The stationary case, in which no change occurs at any point in time, i.e., $\forall t > 0: p^{t-1}(x, y) = p^t(x, y)$, is explicitly excluded from this survey. In such a setting, ODL provides no advantage over the train-then-deploy paradigm: any performance gain could be achieved more reliably by collecting additional labeled data and retraining offline. ODL is therefore only meaningful, and only surveyed here, in the presence of a change in P after T .

3.2 The Application requirements

An application is characterized by a data-generating process P , by the task it addresses (e.g., classification, regression ...), and optionally by two technical requirements on the deployment: a power consumption requirement, i.e., the power drawn by D must remain within a budget imposed by the energy source and form factor of the deployment², and an execution time requirement, i.e., each prediction must be produced within $\bar{E}$ time units of the corresponding observation x^t arriving at D :

$$\forall t > T \quad E(A^t, D) \leq \bar{E}.$$

Satisfying these requirements involves two degrees of freedom: the choice of D and the design of A . The large majority of works in the literature fix D first and optimize the predictive performance of A subject to the resulting constraints³.

The different applications addressed in the literature are analyzed in detail in Section 4.

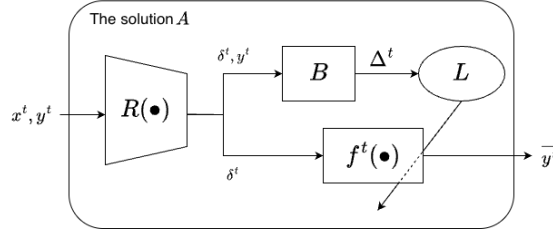
3.3 The Tiny Device D

Let D be the target device on which A is executed, chosen or designed to meet the application requirements introduced above. D is characterized by its RAM capacity M_D , which bounds the amount of data, intermediate activations, and learnable parameters that can reside in memory simultaneously during execution, and by its clock frequency F_D , which jointly determines E alongside the algorithmic complexity of A . Once D is chosen, it imposes its own constraint on A through M_D :

$$M_A^t \leq M_D,$$

which must be satisfied at every time instant $t \geq T$. Note that M_D denotes the total on-chip RAM of the device; the memory actually available to A is strictly smaller, as firmware, the system stack, and runtime overhead consume a non-negligible share of it. Nevertheless, most surveyed works treat M_D as the effective memory budget for the solution. D is further characterized by a flash memory capacity, which determines the amount of static data, such as frozen model parameters and program code, that can be stored on the device. While flash capacity constrains the overall size of A , peak RAM is the binding feasibility constraint for ODL: unlike flash, which is written once at deployment, RAM must simultaneously accommodate the model parameters, the input data, the intermediate activations of the learning mechanism, and the data management buffer during the learning phase. Peak RAM is therefore the primary focus of the memory analysis throughout this survey.

The different devices employed in the literature are analyzed in detail in Section 5.

Fig. 2. The general architecture of an ODL solution A .

3.4 The Components of an ODL Solution

A solution A is composed of four distinct components that communicate with each other as illustrated in Figure 2, regardless of the distribution change regime it is designed to operate in:

- A *dimensionality reduction block* $R(\bullet)$, which takes as input the raw data x^t and outputs a lower-dimensional representation δ^t , where $|\delta^t| \leq |x^t|$;
- A *data management block* B , which selects the instances to be passed to the learning mechanism L . It is characterized by: (i) an admission condition, determining whether a new instance δ^t is stored in an internal buffer Δ^t of maximum capacity $|\Delta|$; (ii) the buffer Δ^t itself, which holds the selected instances and evolves over time as new instances are admitted; and (iii) a trigger condition, determining when the learning mechanism L is activated on the entirety of Δ^t ;
- An *ML algorithm* $f^t(\bullet)$, which takes as input δ^t and outputs the prediction $\bar{y}^t$;
- A *learning mechanism* L , which uses the buffered instances Δ^t to update f^t , with the objective of improving its performance on future instances of S .

At each time instant t , each component has an associated memory requirement and an associated execution time, which together constitute the total memory requirement M_A^t and total execution time E_A^t ⁴ of A at time t , the quantities that must satisfy the constraints introduced in Sections 3.2 and 3.3:

$$M_A^t = M_R^t + M_B^t + M_L^t + M_f^t, \quad (1)$$

$$E_A^t = E_R^t + E_B^t + E_L^t + E_f^t. \quad (2)$$

Without loss of generality, at each time instant t , any component of A can reduce to the identity function, making both its memory requirement and its execution time equal to zero. For instance, a solution with no dimensionality reduction block sets $R(\bullet) = \text{id}(\bullet)$, yielding $M_R^t = E_R^t = 0$. Similarly, any block can be inactive at a given t , reducing its execution time to zero for that instant, while still maintaining a non-zero memory footprint. Finally, both M_A^t and E_A^t may vary over time, depending on the architecture, parameters, and amount of data involved in the operations of each block.

With these edge cases — in particular $R(\bullet) = \text{id}(\bullet)$ and $|\Delta| = 1$ — this decomposition covers all works surveyed in this paper. The boundary between R and f^t is not always sharp in end-to-end trained networks, but the separation is present and meaningful in the large majority of surveyed works.

²Depending on the application, this may apply to average power over a defined operational window, to instantaneous peak power, or both.

³The reverse is also possible: design A first and identify the most efficient or cost-effective D capable of executing it. Co-optimization of both simultaneously is conceivable, but requires explicit weighting of cost, power consumption, and predictive performance against each other.

⁴The additive decomposition assumes sequential execution of the components, which holds for the large majority of the surveyed works. On parallel architectures, components may execute concurrently, in which case E_A^t may need to be computed differently.

The different solutions components employed in the literature are analyzed in detail in Section 6.

3.5 Evaluation of Predictive Performance of ODL Solutions

Evaluating an ODL solution is fundamentally more complex than evaluating a static ML model. In the train-then-deploy paradigm, performance is measured once on a held-out test set after training is complete. In ODL, by contrast, the learning phase occurs during the operational life of D , and performance evolves over time accordingly.

Since the model that will actually run on D does not yet exist at evaluation time, ODL works typically perform evaluation offline and off-device, using a *held-out data stream* that simulates S , constructed from real data withheld for evaluation purposes. Let $m(A, t)$ be a performance metric of A at time t , such as classification accuracy or F1 score. The appropriate evaluation protocol depends on the distribution change regime.

In the single-change regime, A is run on the held-out stream for a given number of steps, and final performance is measured on a separate held-out test set S_{test} drawn from the same post-deployment distribution [64, 71, 75]. This procedure can be repeated across multiple stream lengths to assess data efficiency.

In the concept drift regime, the held-out stream serves as both the source of learning updates and the basis for evaluation, following the prequential protocol [25], in which each sample is first used for inference and then for a learning update [22, 45]. Performance is summarized as a running average of $m(A, t)$ over the stream.

In the continual learning regime, a separate S_{test} is maintained for each distribution encountered in S . Each S_{test} is evaluated both immediately after A finishes learning from the corresponding distribution, and again after exposure to all subsequent distributions [36, 53, 68]. This longitudinal evaluation tracks knowledge acquisition and retention independently: a drop in performance on a previously evaluated S_{test} indicates catastrophic forgetting [56], while an improvement on a not-yet-seen S_{test} indicates positive forward transfer.

4 The Application Lens

This section analyzes the application contexts in which ODL solutions have been deployed and evaluated. The literature is organized along two complementary axes. The primary distinction separates *method-oriented* works, which propose algorithmic contributions validated on standard benchmark datasets without a specific application framing, from *application-oriented* works, which are motivated by and evaluated on a specific real-world deployment scenario. Within each of these two categories, works are further organized by the distribution change regime in which they operate, following the same single-change, concept drift, and continual learning taxonomy established in Section 3.1. Complete end-to-end pipeline implementations, which address label acquisition as a first-class design requirement, are discussed in Section 6.

4.1 Method-Oriented Works

The benchmark datasets used across all method-oriented works are summarized in Table 2, organized by regime and task.

4.1.1 Single-Change Regime. The majority of works evaluate on image classification benchmarks, including CIFAR-10, CIFAR-100, MNIST and their variants, and domain-specific collections such as Flowers, Food, Cars, CUB, and Pets [8, 20, 21, 38, 43, 59, 61, 63, 71, 78–81, 87–89]. A number of these works additionally include audio benchmarks, in particular the Google Speech Commands dataset [61, 70, 78–81] and DCASE2020 [79, 81], as well as Inertial Measurement Units (IMUs) sensor benchmarks for Human Activity Recognition (HAR) such as PAMAP2, SHL, and CWRU [59] and

Table 2. Benchmark datasets used in method-oriented ODL works, organized by regime and task.

Regime	Task	Dataset	Used in
Single-Change	Image Classification	CIFAR-10 [34]	[8, 20, 43, 78–81]
		CIFAR-100 [34]	[8, 43, 81]
		MNIST [40]	[20, 21, 78–81, 87–89]
		Flowers [55], Food [6]	[8, 43, 80, 81]
		Cars [33], CUB [93], Pets [57]	[8, 43]
		Aircraft [49]	[8]
		VWW [13]	[43]
		MiniImageNet [91]	[38]
		Plant Disease [77]	[78]
		Omniglot [39]	[70]
		ImageNet [19]	[21]
	Audio Classification	GSC [95]	[61, 70, 78, 80, 81]
		DCASE2020 [26]	[79, 81]
	IMU-HAR	UCI-HAR [3]	[80, 81]
		PAMAP2 [69], SHL [94], CWRU [85]	[59]
	Tabular	Iris [24]	[87–89]
		Heart Disease [29], Cancer [99]	[88, 89]
Concept Drift	Image Classification	MNIST[40, 100]	[65]
		CIFAR-10-C [27], ImageNet-C [27], SHIFT [90]	[48]
	Audio Classification	GSC [95]	[22]
Continual Learning	Data Stream	DataStream Benchmark Suite [86]	[45]
	Image Classification	CIFAR-10 [34]	[82, 92]
		CIFAR-100 [34]	[36, 97]
		CORE50 [44]	[68, 82, 92]
		MiniImageNet [91]	[36]
		MNIST [40]	[82]
	Audio Classification	GSC [95]	[36, 82]
	IMU-HAR	UCI-HAR [3]	[82]

UCI-HAR [80, 81], reflecting the relevance of these tasks in the TinyML community. Finally, a smaller group of works evaluates on tabular datasets such as Iris, heart disease, and cancer [87–89]. While these datasets are simpler than image or audio benchmarks, they are representative of the sensor data commonly encountered in IoT applications, making them a relevant, even if underused, evaluation domain for TinyML.

4.1.2 Concept Drift Regime. The method-oriented works in the concept drift regime are fewer than in the single-change setting. [22], [48], and [65] evaluate on image and audio benchmarks from the broader machine learning community, including GSC, MNIST, CIFAR-10 and ImageNet, where concept drift is simulated through synthetic corruptions of standard datasets (e.g., CIFAR-10-C, ImageNet-C, SHIFT). In contrast, [45] evaluates exclusively on classic stream learning benchmarks from the data stream mining community, including NOAA, ELEC, RIALTO, POSTURE, COVER, and POKER, which feature naturally occurring distribution shifts rather than synthetic ones.

4.1.3 Continual Learning Regime. The method-oriented works in the continual learning regime evaluate primarily on image classification benchmarks, with CORE50 [68, 82, 92], CIFAR-10 and CIFAR-100 [36, 82, 92, 97], and MiniImageNet [36] being the most commonly used. [82] also includes MNIST as an additional benchmark. CORE50 is particularly well suited to continual learning evaluation as it was specifically designed for object recognition under continuous learning conditions, making it more representative of real deployment scenarios than generic image classification

Table 3. Application-oriented ODL works, organized by regime and task.

Regime	Task	Specific Task	Works	Dataset
Single-Change	KWS and Audio Classification	Speaker adaptation	[14]	GSC [95]
		New keyword enrollment	[11, 73–76]	GSC [95], HeySnips [42], HeySnapdragon [32], MSWC [50], Collected
		Noise adaptation	[15, 16]	GSC [95]
		Speaker verification	[64]	GSC [95], Collected
	Human Pose Estimation	Image: environment adaptation	[9, 10]	Collected
	Human Activity Recognition	IMU: user adaptation	[17]	WISDM [35], Collected
	Biosignal Classification	sEMG: gesture recognition	[5, 7]	UniBo-20-Session [101], Collected
		sEMG: hand dynamics estimation	[102]	HYSER [30]
		ECG: anomaly detection	[1]	MIT-BIH [54]
	Anomaly Detection	Industrial time series	[66]	Collected
IMU: Fan monitoring		[71]	Collected	
Driver behavior		[84]	Collected	
Presence Detection	Multi-sensor: presence detection	[72]	Collected	
Gesture Classification	IMU: on-sensor classification	[12]	Collected	
Concept Drift	Sensor Calibration	IMU: calibration	[62]	Collected, EuRoC MAV
		Pressure sensor: calibration	[83]	Collected
Continual Learning	Biosignal Classification	EEG: BMI personalization	[52, 53]	Collected
	Human Activity Recognition	IMU: class-incremental	[37, 41]	Opportunity, PAMAP2 [69], Skoda, MHEALTH, Collected

benchmarks. Beyond image classification, [36] and [82] additionally evaluate on GSC and UCI-HAR respectively, providing some evidence of generalization beyond the image domain.

4.2 Application-Oriented Works

The application-oriented works surveyed in this section are summarized in Table 3, organized by regime and task.

4.2.1 Single-Change Regime.

Keyword Spotting. Keyword Spotting (KWS) is the most represented application in the single-change regime, addressed from three complementary angles. Personalization to a specific user’s speech characteristics is addressed by [14], which adapts a pre-trained model using a small number of labeled utterances from the target user. Noise adaptation to the acoustic conditions of the deployment context is addressed by [15, 16], which fine-tune a subset of model parameters using recordings collected on-site. The enrollment of new, user-defined keywords is addressed by [11, 73–75], where the challenge is to recognize previously unseen words from a handful of recordings: some works rely on few-shot supervised learning [11, 74, 75], while others eliminate the labeling requirement through self-supervised pseudo-labeling of the incoming audio stream [73]. [64] extend the KWS personalization setting to speaker verification, combining keyword detection with user identity authentication in a two-stage on-device pipeline.

Human Pose Estimation. [9, 10] demonstrate on-device self-supervised fine-tuning for human pose estimation aboard nano-drones, where the model adapts to the visual characteristics of a new deployment context. The supervision signal is derived from a secondary on-board sensor available only during the adaptation phase, making the approach applicable without any human annotation.

Human Activity Recognition. [17] address personalization of IMU-based HAR on STM32 microcontrollers, demonstrating that fine-tuning with a small user-specific labeled dataset significantly reduces inter-subject variability. Both a publicly available benchmark and a proprietary collected dataset are used for evaluation, providing evidence of real-world applicability beyond standard benchmarks.

Biosignal Processing. Physiological signals are inherently person-specific, making them natural candidates for on-device personalization. For ECG anomaly detection, [1] train a personalized model from scratch on the MIT-BIH arrhythmia dataset, a clinically validated benchmark that closely reflects real deployment conditions. For sEMG-based applications, [7] and [5] address gesture recognition personalization, where cross-session variability due to electrode shift is the primary challenge, while [102] extend this to hand dynamics estimation on the HYSER dataset, demonstrating that incremental online training achieves cross-day accuracy comparable to offline training.

Anomaly Detection. Anomaly detection appears across multiple deployment contexts in the single-change regime, sharing the common characteristic that normality must be learned on-device since it cannot be defined a priori. [66] address industrial time series anomaly detection by fitting an analytical model directly from unlabeled post-deployment data. [71] demonstrate on-device anomaly detection for a fan monitoring application, adapting an autoencoder to the normal operating conditions of the deployment platform from collected data. [84] address driver behavior anomaly detection, where normality is user-dependent and learned through unsupervised on-device clustering of vehicle sensor data.

Presence and Occupancy Detection. [72] address occupancy detection from ambient environmental sensors in smart buildings, providing a systematic comparison of online learning algorithms on resource-constrained devices for this task.

Gesture Classification. [12] tackle on-sensor gesture classification in an extremely constrained setting, learning a personalized model from scratch within 8 kB of memory from proprietary collected data.

4.2.2 Concept Drift Regime.

Sensor Calibration. Sensor calibration is the only real-world application addressed in the concept drift regime. Physical sensors exhibit continuous drift over time due to aging, temperature variations, and mechanical stress, making calibration a genuinely non-stationary problem that requires the learning mechanism to remain active throughout the operational life of the device. [62] address IMU calibration under thermal stress, learning a compensation model on the sensor's integrated processor from data collected under controlled temperature variations, with additional validation on the EuRoC MAV dataset under dynamic conditions. [83] address pressure sensor calibration, learning a compensation model from sporadic reference measurements available during deployment, without requiring any offline recalibration procedure.

4.2.3 Continual Learning Regime.

Brain-Machine Interfaces. EEG-based brain-machine interfaces are a natural application for continual learning personalization, as neural signals exhibit strong inter-session variability that causes performance degradation over time, and the set of mental states or motor commands the system must recognize may expand as the user’s needs evolve. [52] and [53] address this by enabling continual on-device adaptation of EEG classification models to the evolving signal characteristics of a specific user, demonstrating that catastrophic forgetting can be mitigated while maintaining the ultra-low power requirements of wearable BMI devices. Both works evaluate on a dataset collected from wearable EEG hardware, with [53] additionally validating on a novel collected dataset.

Human Activity Recognition. Class-incremental learning for HAR addresses the scenario where new activities must be added to the model after deployment without retraining from scratch. [41] address class-incremental HAR from IMU data, evaluating on four publicly available benchmarks including Opportunity, PAMAP2, Skoda, and MHEALTH, and demonstrating resource-efficient expansion of the model as new activity classes are introduced.

4.3 Findings

4.3.1 Method-Oriented vs Application-Oriented Works. A striking observation that emerges from the comparison between method-oriented and application-oriented works is the limited overlap in the tasks and datasets they address. KWS and, to a lesser extent, IMU-based HAR are the only tasks that appear consistently in both categories, with GSC and UCI-HAR serving as their respective shared benchmarks.

This misalignment is more pronounced in TinyML than in conventional machine learning, and reflects two concurrent structural factors. On one side, the application domains that are most studied in the methodological literature, primarily image classification, are too memory and computationally demanding to build relevant real-world TinyML applications with current hardware. On the other side, many of the real-world application domains that do fit within TinyML constraints, such as biosignal processing, sensor calibration, and anomaly detection, lack standardized benchmarks that methodological works can adopt for fair comparison. The result is a field where algorithmic progress and application development are advancing largely in parallel, with limited cross-fertilization.

4.3.2 Personalization, Environmental Adaptation, and Sensor Calibration. The application-oriented works in this survey address three broad categories of real-world challenges. Personalization tasks, the most represented, aim to adapt a model to a specific individual, spanning domains such as KWS, HAR, biosignal processing, and driver behavior analysis. Environmental adaptation tasks address distribution shifts originating from the deployment environment rather than from the user, as in the case of acoustic noise adaptation and visual perception aboard drones. Sensor calibration emerges as a distinct category, particularly in the concept drift regime, where the continuously drifting output of physical sensors must be compensated on-device throughout the operational lifetime of the device.

4.3.3 The Absence of Explicit Application Requirements. Despite the centrality of execution time and power consumption as application-level constraints, as established in Section 3.2, the application-oriented works surveyed here rarely state the requirements they are targeting explicitly. In industrial deployments, power budgets and latency constraints ($\bar{E}$) are strict and non-negotiable. In research works, they are typically left implicit, with the choice of hardware class serving as a proxy. This implicit convention has a structural consequence: without benchmarks that fix an explicit power budget and latency constraint $\bar{E}$, the co-optimization of D and A becomes practically infeasible, and hardware and algorithmic choices are made sequentially rather than jointly. Establishing benchmarks that specify explicit deployment

Table 4. Ultra-Constrained hardware platforms across surveyed ODL works. F_D and M_D report the lower end of each per-work value. Active power figures are estimated from device datasheets; not directly reported in any surveyed work in this class. NR = Not Reported.

Regime	Origin	Core	Works	F_D	M_D	Active Power
Single-Change	Method	Cortex-M0	[20, 87–89]	48– 240 MHz	20–520 kB	10–50 mW
	Application	ISPU	[12]	5–10 MHz	8 kB	~1 mW
		Cortex-M0	[66]	133 MHz	264 kB	20–50 mW
Concept Drift	Method	–	–	–	–	–
	Application	ISPU	[62]	5–10 MHz	8 kB	~1 mW

requirements alongside task and dataset would be a concrete step toward enabling this co-optimization, and is discussed further in Section 7.

5 The Hardware Lens

This section analyzes the hardware platforms on which the surveyed ODL solutions have been deployed and evaluated. As established in Section 3.3, the feasibility of any ODL solution is fundamentally constrained by the memory budget M_D , and by the frequency F_D at which it operates. The hardware choices made across the literature therefore reveal both the current boundaries of what ODL can achieve on real devices and the gap between algorithmic contributions evaluated on capable platforms and solutions deployable on the most constrained hardware.

The hardware platforms reported across the surveyed works can be grouped into three classes based on their computational architecture: *ultra-constrained processors* (Section 5.1), *standard MCUs* (Section 5.2), and *PULP platforms* (Section 5.3). These classes are not merely points on a performance spectrum: each represents a qualitatively different architectural design philosophy, with distinct implications for what ODL solutions are feasible. A non-negligible fraction of the surveyed works do not evaluate on a specific hardware platform, and a smaller number target platforms outside the TinyML scope defined in Section 3.3, such as Cortex-A processors, GPUs, and smartphones. These works are excluded from the hardware analysis but are included in the application and technical analyses where their algorithmic contributions remain relevant.

5.1 Ultra-Constrained Processors

Ultra-constrained processors include platforms lacking a hardware floating-point unit, such as Cortex-M0 based MCUs, with $F_D \leq 100$ MHz, and the ISPU (STMicroelectronics LSM6DSO16IS). The ISPU integrates a hardware FPU but operates at only $F_D = 5$ –10 MHz, placing it in the same practical regime as FPU-less devices. It is also architecturally unique in that it is embedded directly inside the sensor package, eliminating data movement between sensing and processing entirely. The absence of a hardware FPU, combined with a very small M_D , imposes the tightest memory and computational budgets of the three classes and constrains the algorithmic choices available, as discussed in Section 6. Devices in this class span $F_D \in [5 \text{ MHz}, 133 \text{ MHz}]$ (ISPU to RP2040), with M_D ranging from 8 kB to 264 kB, as summarized in Table 4.

Cortex-M0 based boards appear in both method-oriented and application-oriented works. Their widespread availability and low cost make them a natural evaluation target for method contributions seeking to demonstrate feasibility on

Table 5. Standard MCU hardware platforms across surveyed ODL works. F_D and M_D report the lower end of each per-work value. Active power figures are estimated from datasheet active-mode figures; not directly reported in any surveyed work in this class. NR = Not Reported.

Regime	Origin	Core	Works	F_D	M_D	Active Power
Single-Change	Method	Cortex-M4	[31, 58, 61, 70]	180 MHz	250 kB	20–300 mW
		Cortex-M7	[21, 43, 59]	160–216 MHz	64–512 kB	100–500 mW
	Application	Cortex-M4	[1, 17, 64, 71]	64–150 MHz	60–400 kB	20–60 mW
		Cortex-M7	[60]	NR	NR	NR
Concept Drift	Method	Cortex-M4	[65]	64 MHz	256 kB	20–50 mW
		Cortex-M7	[22]	84 MHz	96 kB	300–500 mW
Continual Learning	Method	Cortex-M7	[36]	480 MHz	200 kB	250–600 mW ^a

^aThe upper end (600 mW) marginally exceeds the 500 mW criterion in Section 2.1. The device (STM32H7 at 480 MHz) is MCU-class in all other respects; 600 mW is a worst-case datasheet estimate across supply-voltage configurations, not a measured workload power. This work is included on that basis.

constrained hardware, as well as for application-driven deployments where the algorithmic and power budget is tight. The ISPU, by contrast, appears exclusively in application-oriented works, consistent with its nature as a purpose-built in-sensor processor: its architectural constraints make it unsuitable as a benchmark device for algorithmic contributions, which typically require more flexibility in terms of model architecture and evaluation tooling.

The surveyed works on ultra-constrained hardware are almost entirely concentrated in the single-change regime, with a single exception in concept drift [62] and no works in the continual learning regime. This concentration reflects the constraints of the class: the algorithmic approaches feasible on these platforms can handle a single bounded adaptation phase, but sustaining a continuously active learning mechanism throughout the operational life of D would require memory and computational resources beyond what they offer.

Active power consumption is rarely reported in this class. Based on datasheet figures, the ISPU draws approximately 1 mW in active mode, while Cortex-M0 based boards typically fall in the 10–50 mW range, depending on the specific board and supply voltage.

5.2 Standard MCUs

Standard MCUs include general-purpose single-core processors equipped with a hardware FPU, such as Cortex-M4 and Cortex-M7 based platforms, with $F_D \in [80, 480]$ MHz. These devices follow a conventional scalar pipeline architecture with no hardware-level optimization for neural network workloads, and represent the workhorse class of TinyML, supported by a mature software ecosystem and available across a wide range of off-the-shelf boards. The presence of an FPU and a larger M_D , typically between 256 kB and a few MB, makes backpropagation-based learning feasible on this class, provided that appropriate memory optimizations are applied as discussed in Section 6. Devices in this class span $F_D \in [64, 480]$ MHz, with $M_D \in [60 \text{ kB}, 512 \text{ kB}]$ across the surveyed works.

Standard MCUs are the most balanced class in terms of regime coverage, with works present in all three regimes. The single-change regime is by far the most represented, covering both application-oriented works and methodological contributions exploring backpropagation optimization [31, 43, 58] and forward-only learning [21, 59, 61, 70]. Within this regime, contributions are distributed fairly evenly across Cortex-M4 and Cortex-M7 cores, and between method-oriented and application-oriented works, with no clear concentration in either dimension. In the concept drift regime, standard MCUs are the most represented hardware class [22, 65]: concept drift solutions rely on lightweight incremental algorithms that neither require replay buffers nor incur the backpropagation overhead of continual learning, making

Table 6. PULP hardware platforms across surveyed ODL works. F_D and M_D report the lower end of each per-work value. Active power figures are reported from measurements where available. NR = Not Reported.

Regime	Origin	Device	Works	F_D	M_D	Active Power
Single-Change	Application	GAP8	[7]	100 MHz	32 MB	NR
		GAP9	[9, 10, 16, 73, 75, 76, 102]	240–370 MHz	128 kB–10 MB	19–66 mW
		VEGA	[14, 15]	NR	15 kB–1.5 MB	NR
		Mr. Wolf	[5]	400 MHz	<1 MB	10.4 mW
Continual Learning	Method	GAP9	[97]	240 MHz	10 MB	45 mW
		VEGA	[68]	375 MHz	64 MB	NR
	Application	GAP9	[52, 53]	370 MHz	1–4 MB	21–50 mW

standard MCUs a natural fit. In the continual learning regime, by contrast, only one work targets a standard MCU [36], with most solutions pushed toward more powerful devices by the memory demands of maintaining and replaying past distributions.

Active power figures are not directly reported in any of the surveyed works in this class; all values in Table 5 are estimated from datasheet active-mode figures. Based on these estimates, Cortex-M4 boards typically draw 20–60 mW, while Cortex-M7 boards fall in the 100–600 mW range, reflecting genuine differences in F_D and supply voltage across device families rather than workload variation, as active power is a property of the silicon.

5.3 PULP Platforms

PULP platforms include multi-core parallel ultra-low-power architectures such as the commercial platforms GAP8 and GAP9, and the research prototypes Mr. Wolf and VEGA. What distinguishes this class is not clock speed alone: individual PULP cores have $F_D \in [240, 370]$ MHz, well within the range of standard MCUs. The key difference lies in the architecture: instead of a single general-purpose core, PULP platforms feature a cluster of up to nine small cores operating in parallel over a shared memory. This allows them to process the same workload faster and at a lower energy cost per operation than a single high-performance core. Individual cores span $F_D \in [100, 400]$ MHz and $M_D \in [15 \text{ kB}, 64 \text{ MB}]$ across the surveyed works, reflecting the wide range of deployment scenarios this class supports, as summarized in Table 6.

In the single-change regime, PULP is heavily used for audio and biosignal applications, with GAP9 being the dominant platform [9, 10, 15, 16, 73, 75, 76, 102], alongside GAP8 for an earlier sEMG work [7], VEGA for KWS personalization [14], and Mr. Wolf for EMG gesture recognition [5]. In the concept drift regime, no PULP works were identified. Unlike the absence of ultra-constrained works in continual learning, which can be explained by hardware limitations, the absence of PULP in concept drift is more likely a reflection of the different research communities that have historically addressed these two problems, rather than a fundamental feasibility constraint. The continual learning regime is where PULP platforms are most distinctive: they account for the large majority of continual learning works with reported hardware [52, 53, 68, 97], reflecting the substantially higher memory and computational requirements of continual learning algorithms compared to single-change and concept drift solutions.

Active power is more consistently reported in this class than in the others. GAP9 figures range from 19 to 66 mW during single-change workloads [9, 10, 16, 75] and from 21 to 50 mW for continual learning applications [52, 53], with

[97] reporting 45 mW. Mr. Wolf draws approximately 10.4 mW [5]. VEGA and GAP8 works do not report active power figures.

5.4 Findings

5.4.1 Hardware Capability and Regime Complexity. A clear correlation emerges between hardware capability and regime complexity. Ultra-constrained processors appear almost exclusively in the single-change regime, where the learning phase is bounded and lightweight algorithms suffice. Standard MCUs span all three regimes but are thinly represented in continual learning, where their M_D is often insufficient to sustain an active learning mechanism over the device lifetime. PULP platforms dominate the continual learning regime, being the only class capable of supporting its most demanding solutions. This monotonic relationship suggests that the choice of hardware is not independent of the target regime: more complex regimes impose stricter requirements, and pushing ODL further into non-stationary regimes will likely require hardware capabilities beyond what standard MCUs can currently offer.

A complementary pattern emerges along the method-application axis. General-purpose ARM-based boards, whether Cortex-M0, M4, or M7, are the preferred platform for methodological contributions, favored for their wide availability, low cost, and reproducibility. More specialized platforms, namely the ISPU and PULP devices, appear predominantly in application-oriented works, where the hardware properties are themselves part of the contribution. This divide between algorithmic research, which uses commodity hardware as a neutral substrate, and system-level research, which co-designs the solution with the platform, is a structural feature of the current ODL literature.

5.4.2 Hardware Reporting in the Literature. Comparing hardware platforms in a solution-agnostic way requires metrics that depend solely on D , independently of the algorithmic choices made on top of it. The metrics most commonly reported in the surveyed works, namely peak RAM usage, energy per learning event, and latency, all depend jointly on D and A , as established in Section 3.4, and are therefore unsuitable for this purpose.

We instead characterize each platform by three solution-agnostic metrics: RAM capacity M_D , clock frequency F_D , and peak instantaneous active power consumption, all of which are properties of the silicon rather than of the workload. These metrics describe what a device offers, but not how efficiently it executes a given ODL solution.

This points toward a broader gap: the absence of fixed-solution benchmarks that, by holding A constant and varying D , would make energy, latency, and peak RAM usage comparable across platforms and enable principled hardware selection. This is discussed further as an open challenge in Section 7. The fragmentation of latency and energy reporting conventions observed across the surveyed works is analyzed from the joint solution–hardware perspective in Section 6.3.

5.4.3 PULP Platforms and the Hardware Landscape for ODL. The dominance of PULP platforms in the continual learning regime deserves closer examination, particularly in light of a class of devices that sits between standard MCUs and PULP but has not appeared in the surveyed literature: MCUs equipped with a dedicated neural network accelerator, such as the ARM Cortex-M55 paired with an Ethos-U55 or Ethos-U65 NPU. These platforms retain the single-core scalar architecture of a standard MCU but add a fixed-function hardware block accelerating specific neural network operations, and could in principle offer a middle ground. Their absence from the ODL literature is likely explained by the fact that their software ecosystems have been developed almost exclusively for inference, with no support for gradient computation or on-device training, and that they were not widely available at the time of the earliest works in this survey.

PULP platforms have instead accumulated concrete advantages explaining their prominence. A dedicated ODL research community has formed around GAP9 and VEGA, producing shared methodology and open-source tools such

as DORY and PULP-TrainLib that directly expose training primitives. PULP also offers a contained power envelope with substantially higher computational throughput than standard MCUs, and its flexible architecture, where active cores, memory banks, and off-chip memory can be configured independently, makes it adaptable across a wide range of deployment scenarios.

These advantages are accompanied by significant practical limitations. PULP platforms are considerably more complex to program than standard MCUs, requiring familiarity with cluster execution models, DMA management, and multi-bank memory hierarchies. More fundamentally, no commercial off-the-shelf development kit is currently available, restricting access to research groups with direct connections to the platforms’ developers, and their unit cost remains substantially higher than commodity MCU solutions.

6 The Technical Lens

This section analyzes the technical composition of the ODL solutions surveyed in this work, focusing on how the components described in Section 3.4 are implemented in practice. As argued in Section 3.1, solutions operating under different distribution change regimes are not directly comparable, and the analysis is organized accordingly. The primary axis is the learning paradigm: Section 6.1 covers *batch learning solutions*, in which L executes exactly once on a collected dataset before A transitions permanently to inference, and which by construction are limited to the single-change regime. Section 6.2 covers *incremental learning solutions*, in which L remains active throughout the operational life of D . Within Section 6.2, the secondary axis is the distribution change regime, following the single-change, concept drift, and continual learning taxonomy of Section 3.1.

6.1 Batch Learning Solutions

In batch learning solutions, as illustrated in Figure 3, B first collects a set of N instances from S into the buffer Δ^t . Once the trigger condition fires, L is executed exactly once on Δ^t , producing a definitive version of f^t . The solution then transitions permanently to inference mode, with no further learning taking place. Formally, A goes through three configurations:

- For $t \in [T, T + N - 1]$, only R and B are executed, collecting incoming instances into Δ^t ;
- at $t = T + N$, the trigger condition fires and L is executed on Δ^t , producing a trained f^t ;
- for $t > T + N$, only R and f^t are executed, performing inference on S .

By construction, batch learning solutions have no mechanism for managing further changes in the distribution of P after the initial adaptation phase, and are therefore only applicable in the single-change regime. The surveyed batch learning solutions are summarized in Table 7, and their technical composition is analyzed below.

6.1.1 Dimensionality Reduction Block R . The large majority of works implement R as a pre-trained CNN whose parameters remain frozen throughout the on-device phase [9, 10, 14–17, 64, 73–75]. In audio applications, the CNN is typically preceded by a Mel-frequency cepstral coefficients (MFCC) front-end that converts the raw audio signal into a compact spectral representation before feature extraction [14–16, 64, 73–75]. A smaller group of works foregoes a learned feature extractor entirely: [1] relies on a fixed reservoir followed by PCA, while [66, 87, 88] pass raw or minimally processed data directly to f^t . The works focusing on backpropagation optimization [8, 20, 31, 38, 43, 79–81] set R to the identity function, incorporating the full network into f^t instead. This last pattern is exclusive to method-oriented works: no application-oriented work in the batch learning literature trains end-to-end on-device, reflecting the current practical infeasibility of full network training on constrained hardware for real deployment scenarios.

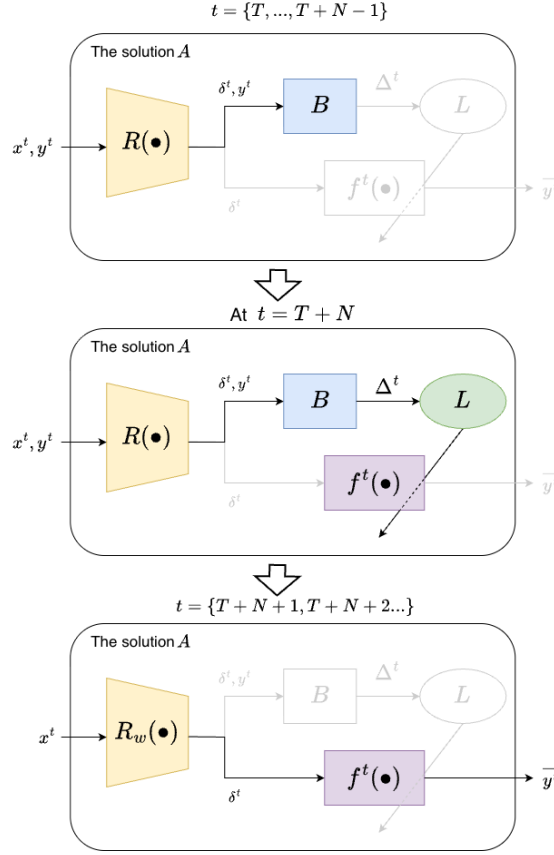


Fig. 3. The sequence of configurations of a batch learning solution A. Colors indicate that the component is active at time t .

6.1.2 Data Management Block B. Across all batch learning solutions, B reduces to a buffer of fixed capacity $|\Delta|$ with a trivial admission condition: every incoming instance is stored until the buffer is full, at which point the trigger condition fires and L is activated. The primary source of variation is in how $|\Delta|$ is determined. The large majority of works assume the complete adaptation dataset fits in memory and set $|\Delta|$ accordingly [1, 8–10, 14–17, 20, 31, 43, 66, 79–81, 87, 88]. A smaller group targets few-shot settings, fixing $|\Delta|$ to a small number of labeled examples per class, typically between one and five [64, 74, 75]. [73] occupies an intermediate position, maintaining a balanced buffer of positive and negative examples whose size is determined by the self-labeling procedure rather than fixed in advance. Similarly, [14–16] retain a portion of S_0 in the buffer alongside new examples, anticipating the replay strategies seen in the continual learning regime.

The memory footprint of B , i.e., $M_B^t = |\Delta| \cdot |\delta^t|$, is frequently the largest term in Equation 1 and is the primary source of underestimation in works that do not target a real deployment device or implement a simplified version of the solution on-device. It is usually not reported in methodological works focused solely on backpropagation optimization [8, 38, 43, 79–81], whose reported memory figures usually compare just the cost of f^t and L with and without the optimization.

Table 7. Batch learning ODL works, organized by origin and device class.

Regime	Origin	Device	Works	R	B	L	f
Single-Change	Method	Ultra- Constr.	[20]	—	All data in mem.	Quant. backprop	Q-DNN
			[87]	—	All data in mem.	SVM train alg.	SVM
			[88]	—	All data in mem.	SGD / 1-v-1	Linear cls.
		Std. MCU	[31]	CNN	All data in mem.	Backprop	DNN
			[43]	CNN	All data in mem.	Sparse backprop	DNN
		No device	[8]	—	All data in mem.	Bias-only backprop	DNN
			[79, 81]	—	All data in mem.	Sparse backprop	DNN
			[38]	CNN	All data in mem.	Sparse backprop	DNN
			[80]	—	All data in mem.	Sparse fwd-fwd	DNN
		Ultra- Constr.	[66]	—	All data in mem.	EM param. fit	Weibull
		Std. MCU	[1]	Reservoir+PCA	All data in mem.	SVM training (SMO)	1-class SVM
			[17]	CNN	All data in mem.	Backprop	DNN
			[64]	MFCC+CNN	Few labeled ex.	Feature storage	KNN-like
		PULP	[9, 10]	CNN	All data in mem.	Backprop	MLP
			[14–16]	MFCC+CNN	New + old data	Backprop	DNN
			[75]	MFCC+CNN	Few ex. per class	Prototype gen.	Prototype
			[73]	MFCC+CNN	Balanced pos./neg.	Backprop + prototypes gen.	Prototype
		No device	[74]	MFCC+CNN	Few ex. per class	Prototype gen.	Prototype

6.1.3 *ML Algorithm f^t and Learning Mechanism L* . The joint choice of f^t and L can be organized into three families, which broadly correspond to the choice of R described above.

The first and most common family pairs a small MLP as f^t with standard backpropagation as L . This combination is natural when R is a frozen pre-trained CNN: the learning phase is restricted to the classification head, keeping both the memory requirement of L and the execution time E_L^t minimal [9, 10, 14–17].

The second family uses a complete neural network as f^t , with R reducing to the identity function. f^t is initialized from a network pre-trained on S_0 and then partially updated on-device, with the associated increase in memory and execution time addressed through an optimized L . Approaches include bias-only updates [8], sparse parameter updates applied selectively to the most important parameters or channels [31, 38, 43, 79, 81], and forward-only learning rules that avoid backpropagation entirely [20, 80].

The third family foregoes neural networks entirely, pairing non-deep classifiers as f^t with learning mechanisms tailored to their structure. Prototype-based classifiers are learned by constructing class representatives from averaged feature vectors [73–75], instance-based classifiers store feature vectors directly for similarity-based classification [64], SVMs are trained through their standard optimization procedure [1, 87], linear classifiers are fitted through SGD with a one-vs-one scheme [88], and parametric models are fitted through expectation-maximization [66]. As noted above, many of these solutions still rely on a frozen pre-trained CNN as R , so that the non-deep nature applies to f^t and L only.

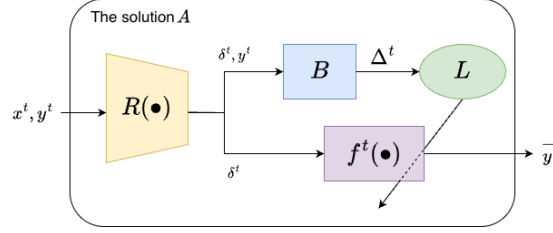


Fig. 4. The configuration of an incremental learning solution A . Colors indicate that the component is active at time t .

6.2 Incremental Learning Solutions

In incremental learning solutions, L is executed multiple times as S progresses, updating f^t every time the trigger condition is satisfied, as illustrated in Figure 4. All four components of A remain active throughout the operational life of D , though not all are required to execute at every time instant t . Unlike batch learning solutions, incremental learning solutions are applicable in all three regimes, as they retain the ability to adapt to further changes in the distribution of P throughout the operational life of D . The surveyed incremental learning solutions are summarized in Table 8, and their technical composition is analyzed below, organized by component.

6.2.1 Dimensionality Reduction Block R . In Incremental Learning, a frozen pre-trained CNN is also the dominant choice for R across all three regimes [7, 21, 22, 36, 52, 53, 59, 61, 63, 65, 68, 70, 71, 78, 92, 97]. This pattern is driven by the constraints of D rather than algorithmic preference: by keeping R frozen, both M_L^t and E_L^t are kept minimal regardless of the regime. In the continual learning regime, this choice is nearly universal, with only [82] and [41] setting R to the identity and training end-to-end on-device. In [36] and [97], R is additionally meta-learned offline to produce representations particularly amenable to few-shot on-device adaptation.

The concept drift regime is the exception to this pattern. The majority of concept drift works set R to the identity function, operating directly on raw or minimally processed sensor data [45, 48, 62, 83], with only [22] and [65] relying on a frozen pre-trained CNN. This reflects the nature of the tasks addressed in that regime: sensor calibration and data stream classification operate on low-dimensional tabular or scalar signals that do not require a learned feature extractor, unlike the audio and image inputs that dominate the single-change and continual learning regimes.

6.2.2 Data Management Block B . The configuration of B reveals the sharpest cross-regime divergence in the incremental learning literature. In the single-change and concept drift regimes, fully online operation with $|\Delta| = 1$ is the dominant strategy, meaning L is executed on each incoming instance individually [12, 21, 45, 62, 70–72, 83, 89, 102]. This choice minimizes the memory footprint of B , keeping $M_B^t \approx 0$, and is consistent with the lightweight algorithmic families that dominate these regimes. A smaller group of single-change works uses explicit buffering: [63] and [78] adopt an active learning strategy selecting only the most informative instances, [59, 61] empty the buffer after each update, and [7] maintains a fixed-capacity buffer with uniform and exponential sampling strategies. In the concept drift regime, [65] maintains a FIFO buffer of fixed capacity, and [22] and [48] activate L only when an explicit change detection mechanism signals a distribution shift.

The continual learning regime stands in sharp contrast: all solutions use explicit buffering, with B collecting instances corresponding to a new distribution before triggering L . This is a structural necessity rather than a design preference, as preventing catastrophic forgetting requires retaining some representation of past distributions alongside new data,

Table 8. Incremental Learning ODL works, organized by environment, origin, and device type. **SC** = Single-Change, **CD** = Concept Drift, **CL** = Continual Learning.

Reg.	Origin	Device	Works	R	B	L	f
Single-Change	Method	Ultra-Const.	[89]	—	Online ($ \Delta =1$)	SGD (binary cls.)	Linear cls.
		Std. MCU	[21]	CNN	Online ($ \Delta =1$)	Feat. vec. store	KNN
			[59, 61]	CNN	Fixed buf., emptied on trigger	Fwd-only (RCE update)	RCE cls.
			[70]	CNN	Online ($ \Delta =1$)	Backprop	DNN
	Appl.	No device	[63, 78]	CNN	Active learning buf. selection	Backprop	DNN
		Ultra-Const.	[12]	Stat. FE	Online ($ \Delta =1$)	Prototype gen.	Prototype
		Std. MCU	[71]	CNN	Online ($ \Delta =1$)	Backprop	DNN
		PULP	[7]	TCN	Fixed buf. (uniform + exp. sampling)	Backprop	TCN
			[102]	—	Online ($ \Delta =1$)	Backprop	TCN
		No device	[72]	—	Online ($ \Delta =1$)	Various	Various
Concept Drift	Method	Std. MCU	[22]	CNN	Fixed buf. + change detect. and removal	Feature vec. storage	KNN-like
			[65]	CNN+Stat. FE	Fixed buf.	Backprop	DNN
		No device	[48]	—	Fixed buf. + change detect.	BN adapt. + model select.	Multi-CNN
			[45]	—	Online ($ \Delta =1$)	Hoeffding-style tree update	Decision tree
	Appl.	Ultra-Const.	[62, 83]	—	Online ($ \Delta =1$)	Fwd-only upd.	RBF-NN
Continual Learning	Method	Std. MCU	[36]	CNN (meta)	Fixed buf. per class	Backprop	DNN
		PULP	[68]	CNN	Fixed buf.	Backprop	DNN
			[97]	CNN (meta)	Fixed buf. per class	Prototype gen.	Prototype
		No device	[92]	CNN	Fixed buf. (binary)	Backprop	BNN
			[82]	—	Fixed buf. (distilled)	Backprop	DNN
	Appl.	PULP	[52]	Filtering	Fixed buf., trigger on acc. drop	Backprop	CNN
			[53]	CNN	Fixed buf.	Backprop	DNN
		No device	[41]	—	Fixed buf.	Backprop + net. expansion	Expandable DNN

in the most common continual learning solutions. The primary sources of variation are in how the buffer content is managed and when L is triggered. To minimize the memory footprint of the replay buffer, [68] and [92] store compact latent representations of past distributions using quantized and binary encodings respectively, while [82] populates Δ^t with synthetic samples obtained through dataset distillation rather than real instances. [36] reduces the buffer requirements from the other direction, using a meta-learned R that produces representations amenable to few-shot adaptation, so that fewer past samples are needed to anchor previous knowledge. Regarding the trigger condition, [52]

Table 9. Component choices for A , with qualitative M_A^t and $E(A^t, D)$ for each component. Categories are relative within each component row: **L** = low, **M** = medium, **H** = high. $\uparrow$ indicates that the metrics grows over time as new data are incorporated. $E(A^t, D)$ refers to inference cost for R and f , and to learning cost for L . B does not contribute to $E(A^t, D)$ directly.

Component	Variant	M_A^t	$E(A^t, D)$
R	Identity ($R = \text{id}$)	L	L
	Statistical FE	L	M
	Frozen pre-trained CNN	H	M-H
B	Online ($ \Delta = 1$)	L	–
	Explicit buffer ($ \Delta > 1$)	M	–
	Full batch collection	H	–
f	Prototype	L $\uparrow$	L $\uparrow$
	KNN	M $\uparrow$	M-H $\uparrow$
	DNN	M-H	M
	CNN	H	H
	TNN	H	H
L	Prototype construction	L	L
	Add to KNN pool	L	L
	Forward-only update	L	M
	Full backpropagation	H	H
	Sparse backpropagation	M	M
	Bias-only / head-only update	L	M

activates L only when classification accuracy drops below an acceptability threshold, while all other works trigger when the buffer is full.

6.2.3 ML Algorithm f^t and Learning Mechanism L . The joint choice of f^t and L follows a clear cross-regime pattern. In the continual learning regime, backpropagation applied to a DNN is the dominant and nearly universal combination [36, 41, 52, 53, 68, 82, 92], with [97] as the only exception, using a prototype-based classifier whose L reduces to constructing a new class representative from averaged feature vectors. The structural reason is straightforward: the explicit replay buffers that all continual learning solutions require make backpropagation the natural learning mechanism to exploit them.

In the single-change and concept drift regimes, the picture is more diverse. Backpropagation-based DNNs remain present in both: [7, 63, 70, 71, 78, 102] in single-change, and [65] and [48] in concept drift. However, lighter alternatives are well represented. Forward-only rules appear in both regimes: [59, 61] pair a hyperspherical RCE classifier with a forward-only update in single-change, while [62] and [83] use RBF networks with a forward-only mechanism in concept drift. Non-neural approaches are also present: KNN-based classifiers in [21] and [22], prototype construction in [12], a binary linear classifier in [89], a Hoeffding-inspired decision tree in [45], and a multi-classifier evaluation in [72].

The concentration of backpropagation in continual learning and its diversification in the other two regimes reflects a structural property discussed further in Section 6.3: non-deep solutions tend to grow in memory over time, making them poorly suited to regimes where A must remain active and adapt indefinitely.

6.3 Findings

6.3.1 *Solution Components, Application Environments, and Hardware classes.* The patterns observed across the surveyed solutions can be understood as consequences of the trade-offs inherent in each component choice, with hardware capability acting as the primary constraint shaping which combinations are feasible. Table 9 summarizes these trade-offs with qualitative estimates of M_A^t and $E(A^t, D)$ for each component choice. The estimates are necessarily qualitative rather than quantitative, for a reason that goes beyond measurement difficulty: no component can be evaluated in isolation from the others. A frozen pre-trained CNN as R produces compact δ^t that allow f to be a simple classifier, whereas identity forces f to absorb the full input complexity. Similarly, full backpropagation may be viable on an ultra-constrained device if f is a single linear layer, yet infeasible if it is a deep CNN. The table should therefore be read as a characterization of the trade-offs each choice introduces, not as an independent ranking of alternatives.

For R , the choice ranges from identity (no overhead, but full input dimensionality exposed to L) through statistical feature extractors (lightweight dimensionality reduction without pre-training) to frozen pre-trained CNNs (highest cost, but compact discriminative representations that directly reduce $|\delta^t|$, the memory footprint of Δ^t , and the labeled data required for adaptation). The frozen CNN is essentially mandatory for image and audio inputs, and is the dominant choice on standard MCUs and PULP platforms. Identity and statistical extractors are more common on ultra-constrained devices, where the cost of running a CNN may be prohibitive, and where simpler signal types are typically used.

For B , fully online operation with $|\Delta| = 1$ minimizes M_B^t and per-update execution time, but limits L to a single pass per instance, making a larger number of labeled instances necessary to reach convergence. Explicit buffering with $|\Delta| > 1$ recovers data efficiency at the cost of a larger M_B^t . Batch collection is the extreme case: peak RAM is highest, as M_B^t and the requirements of L and f^t overlap during the single learning episode, after which the solution runs inference only. Online solutions dominate in single-change and concept drift regimes. Explicit buffering is structurally necessary in continual learning, where past data must be retained to prevent catastrophic forgetting.

For f^t and L , non-deep classifiers are cheaper to update than backpropagation-based networks, but instance-based and prototype-based methods grow in memory as new data are incorporated. Neural networks have a fixed memory footprint regardless of data seen, making them better suited to non-stationary regimes. Within neural network solutions, structured architectures such as RBF and RCE networks reduce learning to a forward pass, while sparsification, quantization, and binarization lower the cost of backpropagation. These optimizations concentrate in the single-change regime, where per-update cost is the primary concern. In continual learning, full backpropagation on capable hardware is the dominant choice, driven by the explicit replay buffers that all solutions in that regime require.

6.3.2 *The Gap Between Methodological and Application-Oriented Solutions.* Two structural gaps emerge when comparing methodological and application-oriented works.

The first concerns scope. Backpropagation optimization, the largest single cluster of methodological contributions in this survey, has produced a rich body of work on sparsification, quantization, and forward-only alternatives, yet most of these contributions are either validated only on standard benchmarks without any on-device implementation, or deployed on standard MCUs under controlled conditions that do not reflect real application constraints. Either way, this work has not translated into application deployments: no application-oriented work across any of the three regimes adopts these optimizations. The reason is not technical incompatibility but a narrowness of focus: methodological works optimize a single component of the pipeline in isolation, typically L , and measure only its cost, without considering that the bottleneck in a real deployment may lie elsewhere, for instance in B or in the labeling strategy. The ODL problem is not solved by optimizing one component: it requires the entire pipeline to be viable simultaneously.

Table 10. Normalization conventions used for latency and energy reporting across the surveyed works, organized by distribution change regime. SC = Single-change, CD = Concept Drift, CL = Continual Learning.

Metric	Unit	SC	CD	CL
Latency	Per instance	[12, 20, 21, 70, 71, 102]	[22]	[53]
	Per batch	[17]	[65]	–
	Per epoch	[9, 10, 43]	–	–
	Per learning event	–	–	[36, 68]
	Per complete run	[1, 5, 7, 15, 16, 31, 64, 73, 75, 87–89]	[62]	[52, 97]
Energy	Per instance	[70, 102]	–	–
	Per batch	[17, 31]	–	–
	Per epoch	[9, 14]	–	–
	Per learning event	–	–	[36, 68]
	Per complete run	[5, 7, 15, 16, 73, 75, 87, 89]	–	[97]

The second gap concerns label availability. Methodological works uniformly assume that labeled data is available in the quantities needed to validate their approach, an assumption that is rarely satisfied in practice, and which is particularly acute in the concept drift and continual learning regimes where application-oriented works remain almost entirely absent. Label availability is often the binding constraint that prevents methodological solutions from being ported to real deployments, and backpropagation optimization and label-efficient learning are rarely developed together despite being complementary concerns. The application-oriented works that have succeeded in real deployments have done so precisely because they treated label acquisition as a first-class design requirement alongside the algorithmic and hardware choices. Four distinct strategies emerge from these works. *Few-shot learning* provides adaptation from a small number of labeled examples supplied by the user, and is adopted by [16, 64, 73, 75] for personalization tasks such as keyword enrollment and speaker verification. *Self-supervised learning with an auxiliary sensor* derives labels automatically from a secondary on-board sensor, eliminating human annotation entirely, as in [9, 10] where supervision comes from the drone’s flight controller during adaptation. *Unsupervised learning* requires no labels at all, and applies naturally to anomaly detection and sensor calibration, where normality is learned from unlabeled post-deployment data [66, 71]. *Implicit labeling through structured stimulus sequences* derives labels from a predefined enrollment protocol rather than explicit annotation, as adopted by [12, 53] for biosignal applications. Each strategy directly constrains the viable solution architecture: few-shot works pair a frozen CNN with a prototype or instance-based classifier, unsupervised works avoid backpropagation entirely, and implicit labeling is specific to settings where an enrollment phase can be designed into the deployment protocol.

Taken together, both gaps point toward the same underlying need: solutions and benchmarks that treat the full pipeline as the unit of design and evaluation, rather than optimizing individual components in isolation. This means co-design frameworks that expose R , B , f , and L as jointly optimizable objects, and application-oriented benchmarks that reflect realistic labeling conditions and deployment constraints, as discussed further in Section 7.

6.3.3 Solution Reporting. Evaluating and comparing ODL solutions is substantially harder than evaluating static ML models. The difficulty manifests across two classes of metrics: joint metrics that depend on both A and D , and solution-only metrics that characterize A independently of hardware.

Joint metrics: latency and energy. Table 10 summarizes the normalization conventions used for latency and energy across the surveyed works. The fragmentation is striking: figures are reported per instance, per batch, per epoch, per learning event, and per complete training run, with no consistency across works or even within the same regime. This makes direct comparison impossible even between solutions targeting the same hardware and task. The problem is not merely cosmetic: a latency reported per complete training run and one reported per instance differ by orders of magnitude for the same solution, depending on dataset size.

Regime-level patterns are visible in Table 10. In the single-change regime, reporting per complete training run dominates for both latency and energy: the learning phase is a one-off event, so total cost is the most operationally relevant figure. In the concept drift regime, energy is entirely absent from the reported metrics, a critical gap given that the cumulative cost of continuously repeating small updates is precisely what matters in this regime; per-instance or per-update figures would be the natural unit yet are rarely used. In the continual learning regime, “per learning event” is the most principled unit, reflecting the episodic structure of task-incremental updates, and the works that do report energy adopt it consistently [36, 68].

Active power in mW, while more self-contained, is insufficient on its own: the same 45 mW device draws very different total energy depending on whether a learning event takes 10 ms or 10 s. A well-reported ODL work should include at minimum latency for a clearly defined learning event and either energy for that event or active power from which it can be derived, with the reference unit stated explicitly.

Solution-only metrics: memory, computational cost, and predictive performance. Peak RAM is the most consistently reported metric, yet many works report only the memory footprint of L and f^t , ignoring M_B^t , which as argued above can be the dominant term in Equation 1. While most surveyed works report hardware-dependent metrics such as execution time and M_D , a minority resort to hardware-independent proxies such as FLOPs, multiply-accumulate operations (MACs), parameter counts, or activation sizes [8, 38, 72, 80, 82, 83, 92], typically when no deployment target D is available. Although these metrics do not directly capture M_D or E on a real D , their hardware independence is a genuine advantage: a worst-case analytical estimate can, for certain classes of solutions, enable fairer cross-platform algorithmic comparison than wall-clock measurements tied to a specific D .

On the predictive performance side, the time axis introduced by ODL makes comparison fundamentally harder than in static ML. In non-stationary regimes, a single accuracy figure is insufficient: a solution that adapts quickly but forgets rapidly is not comparable to one that adapts slowly but retains past knowledge. Even in the single-change regime, the amount of adaptation data is a critical variable that is rarely controlled for: works report results at vastly different numbers of labeled examples per class, making cross-paper comparison unreliable. This is discussed further in Section 7.

7 Future Directions

The analyses carried out in this survey point toward a coherent set of open challenges that, taken together, suggest a broader research agenda for the field.

Standardized evaluation frameworks. A fundamental obstacle to progress in ODL is the absence of standardized evaluation frameworks. As highlighted in Sections 4.3, 5.4, and 6.3, the literature currently lacks consistent evaluation conventions at every level of the analysis. Three complementary frameworks are needed, each addressing a different aspect of the problem.

A *solution-oriented benchmark* should fix the hardware and vary the algorithm, reporting hardware-independent metrics such as worst-case peak RAM, number of operations, and predictive performance across multiple data budgets

to assess label efficiency. Results should be reported across multiple stream lengths to capture the adaptation curve, not just the final accuracy. The full memory budget M_A^t including M_B^t should be reported, and on-device validation should be the norm rather than the exception.

A *hardware-oriented benchmark* should fix the solution and vary the device, reporting solution-agnostic metrics: M_D , F_D , and peak instantaneous active power. This enables direct, reproducible comparison of hardware platforms independently of algorithmic choices, as argued in Section 5.4.

A *co-design benchmark* should report joint metrics (specifically, latency E and energy per learning event) under explicitly stated deployment constraints ($\bar{E}$ and power budget). As argued in Section 4.3, the absence of such benchmarks currently prevents co-optimization of A and D in methodological works, forcing hardware and algorithmic choices to be made sequentially rather than jointly instead.

All three benchmarks should be organized by distribution change regime, since solutions operating under different regimes are not directly comparable, and should be accompanied by clear instructions for dataset construction to enable the community to extend them to new application domains. Real-world application-oriented works, such as those surveyed in Section 4.2, provide the most concrete inspiration for what these benchmarks should look like in practice.

Addressing label efficiency should be an explicit design requirement of the solution-oriented benchmark rather than a separate concern. Concretely, this means constructing data streams that mix labeled and unlabeled instances in realistic proportions, and evaluating solutions on their ability to exploit unlabeled data alongside the scarce labeled examples they receive. Such benchmarks, particularly if designed for the concept drift and continual learning regimes where application-oriented works are currently almost absent, would reflect the actual conditions under which ODL solutions must operate in the real world. This in turn would create the conditions for developing solutions in these regimes that can be transferred to real deployments with greater confidence, rather than remaining validated only on synthetic or laboratory data streams.

Co-design of solution components. As argued in Section 6.3, the surveyed literature predominantly optimizes components of A in isolation, yet the overall efficiency of A depends on all four components jointly. Closing this gap requires not just better benchmarks, but dedicated software tools: libraries and design frameworks that expose all components of A as jointly optimizable objects, support end-to-end evaluation under realistic deployment constraints, and make the cost of each component visible during the design process. No such tool currently exists in the ODL literature, and its development would create the infrastructure needed for principled co-design of R , B , f , and L jointly.

Hardware for ODL. As argued in Section 5.4, the dominant architectural pattern in the surveyed literature keeps the heaviest computation in the frozen R inference pass. Hardware that accelerates inference efficiently is therefore currently more impactful for ODL than hardware that accelerates gradient computation.

Looking further ahead, the constraints of ODL differ fundamentally from those of cloud or edge training: labeled data is scarce, validation cannot happen on-device, power budgets are tight, and the solution must remain operational throughout the learning process. A hardware platform designed with these constraints in mind from the outset, rather than adapted from cloud computing systems architecture, could offer qualitatively different trade-offs. Whether such platforms emerge from extensions of current MCU architectures or from entirely new design paradigms remains an open question.

8 Conclusions

This survey has presented a comprehensive analysis of On-Device Learning for TinyML, organized around a single unifying principle: the distribution change regime that a solution is designed to operate under. By distinguishing between single-change, concept drift, and continual learning regimes, and analyzing each through application, hardware, and technical lenses, the survey provides a framework for comparing ODL solutions in a principled way that prior surveys have not adopted.

From the application lens, a significant structural gap emerges between methodological and application-oriented works. Methodological works cluster around image classification benchmarks too demanding for most real-world TinyML deployments, while the domains that do fit within TinyML constraints lack standardized benchmarks for fair comparison. The works that have succeeded in real deployments share a common characteristic: they treat label acquisition as a first-class design requirement, through few-shot learning, self-supervision, unsupervised approaches, or implicit labeling. Application-oriented works in the concept drift and continual learning regimes remain almost entirely absent, reflecting both the difficulty of obtaining labeled data in non-stationary settings and the absence of realistic benchmarks for these regimes.

From the hardware lens, a clear monotonic correlation emerges between hardware capability and regime complexity: ultra-constrained processors appear almost exclusively in the single-change regime, standard MCUs span all three but are thinly represented in continual learning, and PULP platforms dominate where memory demands are highest. Across the literature, the heaviest computation consistently resides in the frozen feature extractor inference pass, meaning hardware that accelerates inference efficiently is currently more impactful for ODL than hardware designed to accelerate gradient computation.

From the technical lens, the combination of a frozen pre-trained feature extractor and a lightweight learnable classifier is the dominant pattern across all three regimes. Backpropagation optimization concentrates in the single-change regime, while non-stationary regimes rely more heavily on prototype-based classifiers, forward-only learning, and latent replay. A recurring limitation is that components are optimized in isolation, yet the overall efficiency of a solution depends on all four components jointly, and co-optimizing them under realistic deployment constraints remains underexplored.

Taken together, these findings point toward a coherent research agenda. The field will benefit from well defined evaluation frameworks: a solution-oriented benchmark reporting hardware-independent metrics under controlled data budgets, a hardware-oriented benchmark fixing the solution to compare devices on solution-agnostic metrics, and a co-design benchmark specifying explicit deployment constraints to enable joint optimization of solution and device. Beyond benchmarks, the field needs software tools that expose all components of a solution as jointly optimizable objects and support end-to-end evaluation under realistic deployment conditions. This could, in turn, lead to more reliable, more realistic, and more efficient ODL solutions running at the tightest budgets. The framework introduced in this survey, by making the distributional assumptions of each solution explicit and enabling meaningful comparison across the growing body of ODL literature, provides a foundation for this progress.

References

- [1] Norhen Abdennadher, Danilo Pau, and Arcangelo Bruna. 2021. Fixed complexity tiny reservoir heterogeneous network for on-line ECG learning of anomalies. In *2021 IEEE 10th Global Conference on Consumer Electronics (GCCE)*. IEEE, Piscataway, NJ, USA, 233–237. doi:10.1109/GCCE53005.2021.9622022 ISSN: 2378-8143.
- [2] Omar Abozaid, Bassant Selim, and Brigitte Jaumard. 2025. Adaptive Embedded Machine Learning on IoT Devices: A Review. In *2025 International Telecommunications Conference (ITC-Egypt)*. IEEE, Piscataway, NJ, USA, 553–560. doi:10.1109/ITC-Egypt66095.2025.11186607

- [3] D. Anguita, Alessandro Ghio, L. Oneto, Xavier Parra, and Jorge Luis Reyes-Ortiz. 2013. A Public Domain Dataset for Human Activity Recognition using Smartphones. In *The European Symposium on Artificial Neural Networks*. Springer, Bruges (Belgium). <https://api.semanticscholar.org/CorpusID:6975432>
- [4] Colby Banbury, Vijay Janapa Reddi, Peter Torelli, Jeremy Holleman, Nat Jeffries, Csaba Kiraly, Pietro Montino, David Kanter, Sebastian Ahmed, Danilo Pau, et al. 2021. MLPerf Tiny Benchmark. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks* (2021).
- [5] Simone Benatti, Fabio Montagna, Victor Kartsch, Abbas Rahimi, Davide Rossi, and Luca Benini. 2019. Online Learning and Classification of EMG-Based Gestures on a Parallel Ultra-Low Power Platform Using Hyperdimensional Computing. *IEEE Transactions on Biomedical Circuits and Systems* 13, 3 (June 2019), 516–528. doi:10.1109/TBCAS.2019.2914476
- [6] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. 2014. Food-101 – Mining Discriminative Components with Random Forests. In *Computer Vision – ECCV 2014*, David Fleet, Tomas Pajdla, Bernt Schiele, and Tinne Tuytelaars (Eds.). Springer International Publishing, Cham, 446–461.
- [7] Alessio Burrello, Marcello Zanghieri, Cristian Sarti, Leonardo Ravaglia, Simone Benatti, and Luca Benini. 2021. Tackling Time-Variability in sEMG-based Gesture Recognition with On-Device Incremental Learning and Temporal Convolutional Networks. In *2021 IEEE Sensors Applications Symposium (SAS)*. IEEE, Piscataway, NJ, USA, 1–6. doi:10.1109/SAS51076.2021.9530007
- [8] Han Cai, Chuang Gan, Ligeng Zhu, and Song Han. 2021. TinyTL: Reduce Activations, Not Trainable Parameters for Efficient On-Device Learning. doi:10.48550/arXiv.2007.11622 arXiv:2007.11622 [cs].
- [9] Elia Cereda, Alessandro Giusti, and Daniele Palossi. 2024. Training on the Fly: On-device Self-supervised Learning aboard Nano-drones within 20 mW. doi:10.48550/arXiv.2408.03168 arXiv:2408.03168 [cs].
- [10] Elia Cereda, Manuele Rusci, Alessandro Giusti, and Daniele Palossi. 2024. On-device Self-supervised Learning of Visual Perception Tasks aboard Hardware-limited Nano-quadrotors. doi:10.48550/arXiv.2403.04071 arXiv:2403.04071 [cs].
- [11] Jagmohan Chauhan, Young D. Kwon, and Cecilia Mascolo. 2022. Exploring On-Device Learning Using Few Shots for Audio Classification. In *2022 30th European Signal Processing Conference (EUSIPCO)*. IEEE, Piscataway, NJ, USA, 424–428. doi:10.23919/EUSIPCO55093.2022.9909551 ISSN: 2076-1465.
- [12] Mahesh Chowdhary and Swapnil Sayan Saha. 2023. On-Sensor Online Learning and Classification Under 8 KB Memory. In *2023 26th International Conference on Information Fusion (FUSION)*. IEEE, Piscataway, NJ, USA, 1–8. doi:10.23919/FUSION52260.2023.10224228
- [13] Aakanksha Chowdhery, Pete Warden, Jonathon Shlens, Andrew Howard, and Rocky Rhodes. 2019. Visual Wake Words Dataset. arXiv:1906.05721 [cs.CV] <https://arxiv.org/abs/1906.05721>
- [14] Cristian Cioflan, Lukas Cavigelli, and Luca Benini. 2024. Boosting keyword spotting through on-device learnable user speech characteristics. doi:10.48550/arXiv.2403.07802 arXiv:2403.07802 [cs].
- [15] Cristian Cioflan, Lukas Cavigelli, Manuele Rusci, Miguel De Prado, and Luca Benini. 2022. Towards On-device Domain Adaptation for Noise-Robust Keyword Spotting. In *2022 IEEE 4th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. IEEE, Piscataway, NJ, USA, 82–85. doi:10.1109/AICAS54282.2022.9869990
- [16] Cristian Cioflan, Lukas Cavigelli, Manuele Rusci, Miguel de Prado, and Luca Benini. 2024. On-Device Domain Learning for Keyword Spotting on Low-Power Extreme Edge Embedded Systems. doi:10.48550/arXiv.2403.10549 arXiv:2403.10549 [cs].
- [17] Michele Craighero, Davide Quarantiello, Beatrice Rossi, Diego Carrera, Pasqualina Fragneto, and Giacomo Boracchi. 2024. On-Device Personalization for Human Activity Recognition on STM32. *IEEE Embedded Systems Letters* 16, 2 (June 2024), 106–109. doi:10.1109/LES.2023.3293458
- [18] Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. 2021. A continual learning survey: Defying forgetting in classification tasks. *IEEE transactions on pattern analysis and machine intelligence* 44, 7 (2021), 3366–3385.
- [19] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. ImageNet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, Piscataway, NJ, USA, 248–255. doi:10.1109/CVPR.2009.5206848
- [20] Mark Deutel, Frank Hannig, Christopher Mutschler, and Jürgen Teich. 2025. On-Device Training of Fully Quantized Deep Neural Networks on Cortex-M Microcontrollers. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 44, 4 (April 2025), 1250–1261. doi:10.1109/TCAD.2024.3484354
- [21] Simone Disabato and Manuel Roveri. 2020. Incremental On-Device Tiny Machine Learning. In *Proceedings of the 2nd International Workshop on Challenges in Artificial Intelligence and Machine Learning for Internet of Things (AIChallengeIoT '20)*. Association for Computing Machinery, New York, NY, USA, 7–13. doi:10.1145/3417313.3429378
- [22] Simone Disabato and Manuel Roveri. 2024. Tiny Machine Learning for Concept Drift. *IEEE Transactions on Neural Networks and Learning Systems* 35, 6 (June 2024), 8470–8481. doi:10.1109/TNNLS.2022.3229897
- [23] Gregory Ditzler, Manuel Roveri, Cesare Alippi, and Robi Polikar. 2015. Learning in nonstationary environments: A survey. *IEEE Computational Intelligence Magazine* 10, 4 (2015), 12–25.
- [24] R. A. Fisher. 1936. Iris. UCI Machine Learning Repository. doi:10.24432/C56C76
- [25] Joao Gama, Raquel Sebastiao, and Pedro Pereira Rodrigues. 2013. On evaluating stream learning algorithms. *Machine learning* 90 (2013), 317–346.
- [26] Toni Heittola, Annamaria Mesaros, and Tuomas Virtanen. 2020. Acoustic scene classification in DCASE 2020 Challenge: generalization across devices and low complexity solutions. arXiv:2005.14623 [eess.AS] <https://arxiv.org/abs/2005.14623>
- [27] Dan Hendrycks and Thomas G. Dietterich. 2019. Benchmarking Neural Network Robustness to Common Corruptions and Perturbations. *CoRR abs/1903.12261* (2019). arXiv:1903.12261 <http://arxiv.org/abs/1903.12261>

- [28] Maxwell L Hutchinson, Erin Antono, Brenna M Gibbons, Sean Paradiso, Julia Ling, and Bryce Meredig. 2017. Overcoming data scarcity with transfer learning. *arXiv preprint arXiv:1711.05099* (2017).
- [29] Andras Janosi. 1989. Heart Disease. UCI Machine Learning Repository. doi:10.24432/C52P4X
- [30] Xinyu Jiang, Xiangyu Liu, Jiahao Fan, Xinming Ye, Chenyun Dai, Edward A. Clancy, Metin Akay, and Wei Chen. 2021. Open Access Dataset, Toolbox and Benchmark Processing Results of High-Density Surface Electromyogram Recordings. *IEEE Transactions on Neural Systems and Rehabilitation Engineering* 29 (2021), 1035–1046. doi:10.1109/TNSRE.2021.3082551
- [31] Osama Khan, Gwanjong Park, and Euseong Seo. 2023. DaCapo: An On-Device Learning Scheme for Memory-Constrained Embedded Systems. *ACM Trans. Embed. Comput. Syst.* 22, 5s (2023), 142:1–142:23. doi:10.1145/3609121
- [32] Byeongeun Kim, Mingu Lee, Jinkyu Lee, Yeonseok Kim, and Kyuwoong Hwang. 2019. Query-by-example on-device keyword spotting. arXiv:arXiv:1910.05171
- [33] Jonathan Krause, Michael Stark, Jia Deng, and Li Fei-Fei. 2013. 3D Object Representations for Fine-Grained Categorization. In *2013 IEEE International Conference on Computer Vision Workshops*. IEEE, Piscataway, NJ, USA, 554–561. doi:10.1109/ICCVW.2013.77
- [34] Alex Krizhevsky and Geoffrey Hinton. 2009. *Learning multiple layers of features from tiny images*. Technical Report 0. University of Toronto, Toronto, Ontario. <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>
- [35] Jennifer R. Kwapisz, Gary M. Weiss, and Samuel A. Moore. 2011. Activity recognition using cell phone accelerometers. *SIGKDD Explor. Newsl.* 12, 2 (March 2011), 74–82. doi:10.1145/1964897.1964918
- [36] Young D. Kwon, Jagmohan Chauhan, Hong Jia, Stylianos I. Venieris, and Cecilia Mascolo. 2023. LifeLearner: Hardware-Aware Meta Continual Learning System for Embedded Computing Platforms. doi:10.48550/arXiv.2311.11420 arXiv:2311.11420 [cs].
- [37] Young D. Kwon, Jagmohan Chauhan, Abhishek Kumar, Pan Hui HKUST, and Cecilia Mascolo. 2021. Exploring System Performance of Continual Learning for Mobile and Embedded Sensing Applications. In *2021 IEEE/ACM Symposium on Edge Computing (SEC)*. IEEE, Piscataway, NJ, USA, 319–332. doi:10.1145/3453142.3491285
- [38] Young D. Kwon, Rui Li, Stylianos I. Venieris, Jagmohan Chauhan, Nicholas D. Lane, and Cecilia Mascolo. 2024. TinyTrain: Resource-Aware Task-Adaptive Sparse Training of DNNs at the Data-Scarce Edge. doi:10.48550/arXiv.2307.09988 arXiv:2307.09988 [cs].
- [39] Brenden M. Lake, Ruslan Salakhutdinov, and Joshua B. Tenenbaum. 2015. Human-level concept learning through probabilistic program induction. *Science* 350, 6266 (2015), 1332–1338. arXiv:https://www.science.org/doi/pdf/10.1126/science.aab3050 doi:10.1126/science.aab3050
- [40] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324. doi:10.1109/5.726791
- [41] Clayton Frederick Souza Leite and Yu Xiao. 2022. Resource-Efficient Continual Learning for Sensor-Based Human Activity Recognition. *ACM Trans. Embed. Comput. Syst.* 21, 6 (2022), 85:1–85:25. doi:10.1145/3530910
- [42] David Leroy, Alice Coucke, Thibaut Lavril, Thibault Gisselbrecht, and Joseph Dureau. 2019. Federated Learning for Keyword Spotting. arXiv:1810.05512 [eess.AS] <https://arxiv.org/abs/1810.05512>
- [43] Ji Lin, Ligeng Zhu, Wei-Ming Chen, Wei-Chen Wang, Chuang Gan, and Song Han. 2024. On-Device Training Under 256KB Memory. doi:10.48550/arXiv.2206.15472 arXiv:2206.15472 [cs].
- [44] Vincenzo Lomonaco and Davide Maltoni. 2017. CORE50: a New Dataset and Benchmark for Continuous Object Recognition. In *Proceedings of the 1st Annual Conference on Robot Learning (Proceedings of Machine Learning Research, Vol. 78)*, Sergey Levine, Vincent Vanhoucke, and Ken Goldberg (Eds.). PMLR, 17–26. <https://proceedings.mlr.press/v78/lomonaco17a.html>
- [45] Afonso Lourenço, João Rodrigo, João Gama, and Goreti Marreiros. 2025. DFDT: Dynamic Fast Decision Tree for IoT Data Stream Mining on Edge Devices. doi:10.48550/arXiv.2502.14011 arXiv:2502.14011 [cs].
- [46] Afonso Lourenço, João Rodrigo, João Gama, and Goreti Marreiros. 2025. On-device edge learning for IoT data streams: a survey. doi:10.48550/arXiv.2502.17788 arXiv:2502.17788 [cs].
- [47] Jie Lu, Anjin Liu, Fan Dong, Feng Gu, Joao Gama, and Guangquan Zhang. 2018. Learning under concept drift: A review. *IEEE transactions on knowledge and data engineering* 31, 12 (2018), 2346–2363.
- [48] Xiao Ma, Young D. Kwon, and Dong Ma. 2025. On-demand Test-time Adaptation for Edge Devices. doi:10.48550/arXiv.2505.00986 arXiv:2505.00986 [cs].
- [49] S. Maji, J. Kannala, E. Rahtu, M. Blaschko, and A. Vedaldi. 2013. *Fine-Grained Visual Classification of Aircraft*. Technical Report. arXiv:1306.5151 [cs-cv]
- [50] Mark Mazumder, Sharad Chitlangia, Colby Banbury, Yiping Kang, Juan Manuel Ciro, Keith Achorn, Daniel Galvez, Mark Sabini, Peter Mattson, David Kanter, Greg Diamos, Pete Warden, Josh Meyer, and Vijay Janapa Reddi. 2021. Multilingual Spoken Words Corpus. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*. <https://openreview.net/forum?id=c20jj5K2H>
- [51] Julian McAuley. 2022. *Personalized Machine Learning*. Cambridge University Press.
- [52] Lan Mei, Cristian Cioflan, Thorir Mar Ingolfsson, Victor Kartsch, Andrea Cossettini, Xiaying Wang, and Luca Benini. 2024. Train-On-Request: An On-Device Continual Learning Workflow for Adaptive Real-World Brain Machine Interfaces. In *2024 IEEE Biomedical Circuits and Systems Conference (BioCAS)*. IEEE, Piscataway, NJ, USA, 1–5. doi:10.1109/BioCAS61083.2024.10798357 ISSN: 2766-4465.
- [53] Lan Mei, Thorir Mar Ingolfsson, Cristian Cioflan, Victor Kartsch, Andrea Cossettini, Xiaying Wang, and Luca Benini. 2025. An Ultra-Low Power Wearable BMI System With Continual Learning Capabilities. *IEEE Transactions on Biomedical Circuits and Systems* 19, 3 (June 2025), 511–522. doi:10.1109/TBCAS.2024.3457522

- [54] G.B. Moody and R.G. Mark. 2001. The impact of the MIT-BIH arrhythmia database. *IEEE engineering in medicine and biology magazine : the quarterly magazine of the Engineering in Medicine & Biology Society* 20 (06 2001), 45–50. doi:10.1109/51.932724
- [55] Maria-Elena Nilsback and Andrew Zisserman. 2008. Automated Flower Classification over a Large Number of Classes. In *2008 Sixth Indian Conference on Computer Vision, Graphics & Image Processing*. 722–729. doi:10.1109/ICVGIP.2008.47
- [56] German I Parisi, Ronald Kemker, Jose L Part, Christopher Kanan, and Stefan Wermter. 2019. Continual lifelong learning with neural networks: A review. *Neural networks* 113 (2019), 54–71.
- [57] Omkar M Parkhi, Andrea Vedaldi, Andrew Zisserman, and C. V. Jawahar. 2012. Cats and dogs. In *2012 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, Piscataway, NJ, USA, 3498–3505. doi:10.1109/CVPR.2012.6248092
- [58] Shishir G. Patil, Paras Jain, Prabal Dutta, Ion Stoica, and Joseph E. Gonzalez. 2022. POET: Training Neural Networks on Tiny Devices with Integrated Rematerialization and Paging. doi:10.48550/arXiv.2207.07697 arXiv:2207.07697 [cs].
- [59] Danilo Pau, Prem Kumar Ambrose, Andrea Pisani, and Fabrizio M. Aymone. 2023. TinyRCE: Forward Learning Under Tiny Constraints. In *2023 IEEE International Conference on Metrology for eXtended Reality, Artificial Intelligence and Neural Engineering (MetroXRINE)*. IEEE, Piscataway, NJ, USA, 295–300. doi:10.1109/MetroXRINE58569.2023.10405784
- [60] Danilo Pau, Abderrahim Khiari, and Davide Denaro. 2021. Online learning on tiny micro-controllers for anomaly detection in water distribution systems. In *2021 IEEE 11th International Conference on Consumer Electronics (ICCE-Berlin)*. IEEE, Piscataway, NJ, USA, 1–6. doi:10.1109/ICCE-Berlin53567.2021.9720009 ISSN: 2166-6822.
- [61] Danilo Pietro Pau, Andrea Pisani, Fabrizio Maria Aymone, and Gianluigi Ferrari. 2023. TinyRCE: Multipurpose Forward Learning for Resource Restricted Devices. *IEEE Sensors Letters* 7, 10 (Oct. 2023), 1–4. doi:10.1109/LSENS.2023.3307119
- [62] Danilo Pietro Pau, Simone Tognocchi, and Marco Marcon. 2025. Learning Online MEMS Calibration with Time-Varying and Memory-Efficient Gaussian Neural Topologies. *Sensors* 25, 12 (Jan. 2025), 3679. doi:10.3390/s25123679 Publisher: Multidisciplinary Digital Publishing Institute.
- [63] Massimo Pavan, Claudio Galimberti, and Manuel Roveri. 2025. TactiLE: Tiny Active LEarning for wearable devices. In *2025 International Joint Conference on Neural Networks (IJCNN)*. IEEE, Piscataway, NJ, USA, 1–8.
- [64] Massimo Pavan, Gioele Mombelli, Francesco Sinacori, and Manuel Roveri. 2025. TinySV: Speaker Verification in TinyML with On-device Learning. In *Proceedings of the 4th International Conference on AI-ML Systems (AIMLSys '24)*. Association for Computing Machinery, New York, NY, USA, 1–10. doi:10.1145/3703412.3703415
- [65] Massimo Pavan, Eugeniu Ostrovan, Armando Caltabiano, and Manuel Roveri. 2024. TyBox: An Automatic Design and Code Generation Toolbox for TinyML Incremental On-Device Learning. *ACM Trans. Embed. Comput. Syst.* 23, 3 (2024), 42:1–42:27. doi:10.1145/3604566
- [66] Eduardo S. Pereira, Leonardo S. Marcondes, and Josemar M. Silva. 2023. On-Device Tiny Machine Learning for Anomaly Detection Based on the Extreme Values Theory. *IEEE Micro* 43, 6 (Nov. 2023), 58–65. doi:10.1109/MM.2023.3316918
- [67] Visal Rajapakse, Ishan Karunanayake, and Nadeem Ahmed. 2023. Intelligence at the Extreme Edge: A Survey on Reformable TinyML. *ACM Comput. Surv.* 55, 13s (2023), 282:1–282:30. doi:10.1145/3583683
- [68] Leonardo Ravaglia, Manuele Rusci, Davide Nadalini, Alessandro Capotondi, Francesco Conti, and Luca Benini. 2021. A TinyML Platform for On-Device Continual Learning with Quantized Latent Replays. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 11, 4 (Dec. 2021), 789–802. doi:10.1109/JETCAS.2021.3121554 arXiv:2110.10486 [cs].
- [69] Attila Reiss and Didier Stricker. 2012. Introducing a New Benchmarked Dataset for Activity Monitoring. *2012 16th International Symposium on Wearable Computers* (2012), 108–109. https://api.semanticscholar.org/CorpusID:10337279
- [70] Haoyu Ren, Darko Anicic, Xue Li, and Thomas Runkler. 2024. On-device Online Learning and Semantic Management of TinyML Systems. *ACM Trans. Embed. Comput. Syst.* 23, 4 (2024), 55:1–55:32. doi:10.1145/3665278
- [71] Haoyu Ren, Darko Anicic, and Thomas Runkler. 2021. TinyOL: TinyML with Online-Learning on Microcontrollers. doi:10.48550/arXiv.2103.08295 arXiv:2103.08295 [cs].
- [72] Flavio Renzi, Haoyu Ren, Alessio Bernardo, Giacomo Ziffer, Darko Anicic, and Emanuele Della Valle. 2025. Online Learning Techniques for Occupancy Detection on Resource Constrained Devices. In *2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. IEEE, Piscataway, NJ, USA, 1019–1026. doi:10.1109/IPDPSW66978.2025.00159 ISSN: 2995-066X.
- [73] Manuele Rusci, Francesco Paci, Marco Fariselli, Eric Flamand, and Tinne Tuytelaars. 2025. Self-Learning for Personalized Keyword Spotting on Ultra-Low-Power Audio Sensors. *IEEE Internet of Things Journal* 12, 8 (April 2025), 10210–10221. doi:10.1109/JIOT.2024.3515143 arXiv:2408.12481 [cs].
- [74] Manuele Rusci and Tinne Tuytelaars. 2023. Few-Shot Open-Set Learning for On-Device Customization of KeyWord Spotting Systems. doi:10.48550/arXiv.2306.02161 arXiv:2306.02161 [cs].
- [75] Manuele Rusci and Tinne Tuytelaars. 2023. On-Device Customization of Tiny Deep Learning Models for Keyword Spotting With Few Examples. *IEEE Micro* 43, 6 (Nov. 2023), 50–57. doi:10.1109/MM.2023.3311826
- [76] Manuele Rusci, Hugo Van Hamme, and Tinne Tuytelaars. 2025. Self-Incremental Training for Personalized Voice Command Recognition in a Wireless Audio Sensor Network. In *ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, Piscataway, NJ, USA, 1–5. doi:10.1109/ICASSP49660.2025.10887592 ISSN: 2379-190X.
- [77] J. Ruth, Uma Ranganathan, Meenakshi Ayyathurai, and Pooja Ramkumar. 2022. Meta-Heuristic Based Deep Learning Model for Leaf Diseases Detection. *Neural Processing Letters* 54 (06 2022). doi:10.1007/s11063-022-10880-z

- [78] Marcus Rüb, Daniel Konegen, Patrick Selle, Axel Sikora, and Daniel Mueller-Gritschneider. 2025. DRIP: DRop unImportant data Points – Enhancing Machine Learning Efficiency with Grad-CAM-Based Real-Time Data Prioritization for On-Device Training. [doi:10.48550/arXiv.2504.08364](https://doi.org/10.48550/arXiv.2504.08364) arXiv:2504.08364 [cs].
- [79] Marcus Rüb, Daniel Maier, Daniel Mueller-Gritschneider, and Axel Sikora. 2023. TinyProp – Adaptive Sparse Backpropagation for Efficient TinyML On-device Learning. [doi:10.48550/arXiv.2308.09201](https://doi.org/10.48550/arXiv.2308.09201) arXiv:2308.09201 [cs].
- [80] Marcus Rüb, Michael Rüb, and Axel Sikora. 2025. Sparse Forward-Forward Algorithm: Efficient Machine Learning for Resource-Constrained Edge Devices. In *2025 IEEE 8th International Conference on Industrial Cyber-Physical Systems (ICPS)*. IEEE, Piscataway, NJ, USA, 1–6. [doi:10.1109/ICPS65515.2025.11087822](https://doi.org/10.1109/ICPS65515.2025.11087822) ISSN: 2769-3899.
- [81] Marcus Rüb, Axel Sikora, and Daniel Mueller-Gritschneider. 2024. Advancing On-Device Neural Network Training with TinyPropv2: Dynamic, Sparse, and Efficient Backpropagation. In *2024 International Joint Conference on Neural Networks (IJCNN)*. IEEE, Piscataway, NJ, USA, 1–8. [doi:10.1109/IJCNN60899.2024.10650122](https://doi.org/10.1109/IJCNN60899.2024.10650122) arXiv:2409.07109 [cs].
- [82] Marcus Rüb, Philipp Tüchel, Axel Sikora, and Daniel Mueller-Gritschneider. 2024. A Continual and Incremental Learning Approach for TinyML On-device Training Using Dataset Distillation and Model Size Adaption. In *2024 IEEE 7th International Conference on Industrial Cyber-Physical Systems (ICPS)*. IEEE, Piscataway, NJ, USA, 1–8. [doi:10.1109/ICPS59941.2024.10639989](https://doi.org/10.1109/ICPS59941.2024.10639989) arXiv:2409.07114 [cs].
- [83] Francesco Saccani, Danilo Pau, and Michele Amoretti. 2024. In-Sensor Learning for Pressure Self-Calibration. In *2024 IEEE Sensors Applications Symposium (SAS)*. IEEE, Piscataway, NJ, USA, 1–6. [doi:10.1109/SAS60918.2024.10636625](https://doi.org/10.1109/SAS60918.2024.10636625) ISSN: 2766-3078.
- [84] Marianne Silva, Thais Medeiros, Mariana Azevedo, Morsinaldo Medeiros, Mikael Themoteo, Tatiane Gois, Ivanovitch Silva, and Daniel G. Costa. 2023. An Adaptive TinyML Unsupervised Online Learning Algorithm for Driver Behavior Analysis. In *2023 IEEE International Workshop on Metrology for Automotive (MetroAutomotive)*. IEEE, Piscataway, NJ, USA, 199–204. [doi:10.1109/MetroAutomotive57488.2023.10219125](https://doi.org/10.1109/MetroAutomotive57488.2023.10219125)
- [85] W.A. Smith and R.B. Randall. 2015. Case Western Reserve University Bearing Data Center. <https://engineering.case.edu/bearingdatacenter/download-data-file>. Accessed: [Insert Date Here].
- [86] Vinicius M. A. Souza, Denis M. dos Reis, André G. Maletzke, and Gustavo E. A. P. A. Batista. 2020. Challenges in benchmarking stream learning algorithms with real-world data. *Data Mining and Knowledge Discovery* 34, 6 (July 2020), 1805–1858. [doi:10.1007/s10618-020-00698-5](https://doi.org/10.1007/s10618-020-00698-5)
- [87] Bharath Sudharsan, John G. Breslin, and Muhammad Intizar Ali. 2020. Edge2Train: a framework to train machine learning models (SVMs) on resource-constrained IoT edge devices. In *Proceedings of the 10th International Conference on the Internet of Things (IoT '20)*. Association for Computing Machinery, New York, NY, USA, 1–8. [doi:10.1145/3410992.3411014](https://doi.org/10.1145/3410992.3411014)
- [88] Bharath Sudharsan, John G. Breslin, and Muhammad Intizar Ali. 2022. ML-MCU: A Framework to Train ML Classifiers on MCU-Based IoT Edge Devices. *IEEE Internet of Things Journal* 9, 16 (Aug. 2022), 15007–15017. [doi:10.1109/JIOT.2021.3098166](https://doi.org/10.1109/JIOT.2021.3098166)
- [89] Bharath Sudharsan, Piyush Yadav, John G. Breslin, and Muhammad Intizar Ali. 2021. Train+: An Incremental ML Model Training Algorithm to Create Self-Learning IoT Devices. In *2021 IEEE SmartWorld, Ubiquitous Intelligence & Computing, Advanced & Trusted Computing, Scalable Computing & Communications, Internet of People and Smart City Innovation (SmartWorld/SCALCOM/UIC/ATC/IOP/SCI)*. IEEE, Piscataway, NJ, USA, 97–106. [doi:10.1109/SWC50871.2021.00023](https://doi.org/10.1109/SWC50871.2021.00023)
- [90] Tao Sun, Mattia Segu, Janis Postels, Yuxuan Wang, Luc Van Gool, Bernt Schiele, Federico Tombari, and Fisher Yu. 2022. SHIFT: A Synthetic Driving Dataset for Continuous Multi-Task Domain Adaptation. arXiv:2206.08367 [cs.CV] <https://arxiv.org/abs/2206.08367>
- [91] Oriol Vinyals, Charles Blundell, Timothy Lillicrap, Koray Kavukcuoglu, and Daan Wierstra. 2016. Matching networks for one shot learning. In *Proceedings of the 30th International Conference on Neural Information Processing Systems (Barcelona, Spain) (NIPS'16)*. Curran Associates Inc., Red Hook, NY, USA, 3637–3645.
- [92] Lorenzo Vorabbi, Davide Maltoni, Guido Borghi, and Stefano Santi. 2024. Enabling On-Device Continual Learning with Binary Neural Networks and Latent Replay. In *Proceedings of the 19th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications*. SCITEPRESS - Science and Technology Publications, Rome, Italy, 25–36. [doi:10.5220/0012269000003660](https://doi.org/10.5220/0012269000003660)
- [93] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie. 2011. *CUB Dataset*. Technical Report CNS-TR-2011-001. California Institute of Technology.
- [94] Hao Wang, Huazhen Huang, Jinfeng Wang, and Fangmin Sun. 2024. Sussex-Huawei Locomotion Recognition Using Machine Learning and Deep Learning with Multi-sensor data. In *Companion of the 2024 on ACM International Joint Conference on Pervasive and Ubiquitous Computing (Melbourne VIC, Australia) (UbiComp '24)*. Association for Computing Machinery, New York, NY, USA, 563–568. [doi:10.1145/3675094.3678457](https://doi.org/10.1145/3675094.3678457)
- [95] Pete Warden. 2018. Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition. arXiv:1804.03209 [cs] (April 2018). <http://arxiv.org/abs/1804.03209> arXiv: 1804.03209.
- [96] Pete Warden and Daniel Situnayake. 2020. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-low-power Microcontrollers*. O'Reilly. Google-Books-ID: sB3mxQEACAAJ.
- [97] Yoga Esa Wibowo, Cristian Cioflan, Thorir Mar Ingolfsson, Michael Hersche, Leo Zhao, Abbas Rahimi, and Luca Benini. 2024. 12 mJ Per Class On-Device Online Few-Shot Class-Incremental Learning. In *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, Piscataway, NJ, USA, 1–6. [doi:10.23919/DAT58400.2024.10546598](https://doi.org/10.23919/DAT58400.2024.10546598) ISSN: 1558-1101.
- [98] Gerhard Widmer and Miroslav Kubat. 1996. Learning in the presence of concept drift and hidden contexts. *Machine learning* 23 (1996), 69–101.
- [99] William Wolberg. 1993. Breast Cancer Wisconsin (Diagnostic). UCI Machine Learning Repository. [doi:10.24432/C5DW2B](https://doi.org/10.24432/C5DW2B)
- [100] Han Xiao, Kashif Rasul, and Roland Vollgraf. 2017. Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. arXiv:1708.07747 [cs.LG] <https://arxiv.org/abs/1708.07747>

- [101] Marcello Zanghieri, Simone Benatti, Alessio Burrello, Victor Kartsch, Francesco Conti, and Luca Benini. 2020. Robust Real-Time Embedded EMG Recognition Framework Using Temporal Convolutional Networks on a Multicore IoT Processor. *IEEE Transactions on Biomedical Circuits and Systems* 14, 2 (2020), 244–256. doi:10.1109/TBCAS.2019.2959160
- [102] Marcello Zanghieri, Pierangelo Maria Rapa, Mattia Orlandi, Elisa Donati, Luca Benini, and Simone Benatti. 2024. sEMG-Driven Hand Dynamics Estimation With Incremental Online Learning on a Parallel Ultra-Low-Power Microcontroller. *IEEE Transactions on Biomedical Circuits and Systems* 18, 4 (Aug. 2024), 810–820. doi:10.1109/TBCAS.2024.3415392

Acknowledgments

The authors used Claude Sonnet 4.6 (Anthropic) as an AI-assisted writing tool to support text editing and revision during the preparation of this manuscript. All outputs were reviewed, revised, and validated by the authors, who take full responsibility for the content.

9 Mathematical Notation

Table 11 lists all mathematical symbols used throughout the paper.

Table 11. Mathematical notation used throughout the paper.

Symbol	Meaning
<i>Problem formulation</i>	
P	Data-generating process
D	Target device
A	ODL solution
T	Deployment time
S_0	Pre-deployment dataset
S_{test}	Held-out test set
S	Post-deployment data stream
<i>Device and application constraints</i>	
M_D	Device RAM capacity
M_A^t	RAM used by A at time t
F_D	Device clock frequency
$\frac{E}{A}^t$	Execution time of A at time t
E	Latency constraint
<i>Solution components</i>	
R	Dimensionality reduction block
δ^t	Latent representation at time t
B	Data management block
Δ^t	Data management buffer at time t
f^t	ML algorithm at time t
L	Learning mechanism
<i>Data and evaluation</i>	
x^t	Input observation at time t
y^t	Prediction at time t
m	Performance metric
N	Adaptation set size

Received –; revised –; accepted –

Manuscript submitted to ACM